

Python und MicroPython als standardisierte Programmiersprache in AUTOSAR

Stephan Epp

26. März 2026

Inhaltsverzeichnis

1	Einleitung	4
2	AUTOSAR – Architektur und Anforderungen	5
2.1	Geschichte und Motivation	5
2.2	AUTOSAR Classic Platform (CP)	5
2.3	AUTOSAR Adaptive Platform (AP)	6
2.4	Anforderungskatalog an eine Programmiersprache in AUTOSAR	6
3	Python und MicroPython – Überblick und Eigenschaften	7
3.1	Python – Die universelle Hochsprache	7
3.1.1	Statische Typprüfung und Verifikation	7
3.1.2	CPython und alternative Implementierungen	8
3.2	MicroPython – Python für Mikrocontroller	8
3.2.1	Unterstützte Mikrocontroller-Plattformen	9
3.2.2	MicroPython Performance	9
3.2.3	MicroPython Speicherverwaltung im Detail	9
4	Analyse der Rust-Standardisierungsbewegung in AUTOSAR	10
4.1	Die Rust-Bewegung im Überblick	10
4.2	Kritische Analyse der Rust-Eignung für AUTOSAR	10
4.2.1	Lernkurve und Entwicklerengpass	10
4.2.2	Fehlendes Safety-Zertifizierungs-Ökosystem	11
4.2.3	Fehlende AUTOSAR-Treiberintegration	11
4.2.4	‘unsafe’ Rust: kein Gewinn gegenüber C	11
4.2.5	Zusammenfassung: Rust vs. Python/MicroPython	11
5	Technische Eignung von Python/MicroPython für AUTOSAR	11
5.1	Erfüllung des Anforderungskatalogs	11
5.2	MicroPython auf Automotive-Mikrocontrollern	13
5.3	Latenz und Interrupt-Handling	14

6	Sicherheitstechnische Betrachtung	14
6.1	Functional Safety nach ISO 26262	14
6.1.1	Anforderungen an die Sprachlaufzeit	14
6.1.2	PySafety-Sprachprofil	15
6.1.3	Coding Guidelines und statische Prüfung	15
6.2	Cybersecurity nach ISO 21434	15
6.3	WCET-Analyse und Echtzeitfähigkeit	16
7	Integrationsarchitektur	17
7.1	Überblick: Python in der AUTOSAR-Schichtenarchitektur	17
7.2	Python SWC Integration – Classic Platform	17
7.2.1	PyAutosar-RTE-Bridge	17
7.2.2	MicroPython-VM Integration in AUTOSAR-OS	18
7.3	Python Integration – Adaptive Platform	19
7.4	pyautosar – Existierende Python/AUTOSAR-Tooling-Infrastruktur	20
7.5	OTA-Update via Python	20
8	Das Python-Ökosystem im Automotive-Kontext	21
8.1	Weltgrößtes Entwickler-Ökosystem	21
8.2	Testautomatisierung	21
8.3	KI/ML-Integration	22
8.4	Diagnose und Engineering Tools	23
9	Wirtschaftliche Analyse	23
9.1	Entwicklerproduktivität	23
9.2	Talentpool und Rekrutierung	24
9.3	Reduzierte Toolkost	24
10	Standardisierungsvorschlag	24
10.1	Einbettung in den AUTOSAR-Standardisierungsprozess	24
10.1.1	Phase 1: Bildung einer Working Group (2026–2027)	24
10.1.2	Phase 2: Spezifikation (2027–2028)	25
10.1.3	Phase 3: Pilotprojekte und Validierung (2028–2029)	25
10.1.4	Phase 4: Normative Aufnahme (2029–2030)	25
10.2	Technischer Standard: AUTOSAR_SWS_PythonRuntime (Entwurf)	25
10.3	Referenz-Implementierung: PyAutosar	26
11	Verwandte Arbeiten	26
11.1	Python in eingebetteten Systemen	26
11.2	Sprachsicherheit und Safety	27
11.3	AUTOSAR-Spracherweiterungen	27
12	Diskussion und Ausblick	27
12.1	Zusammenfassung der Ergebnisse	27
12.2	Offene Forschungsfragen	28
12.2.1	Echtzeit und WCET-Analyse	28
12.2.2	Interpreter-Qualifizierung	28
12.2.3	PySafety-Profil und statische Analyse	29

12.2.4	Performance und Portierung	29
12.2.5	Sicherheit und Isolation	30
12.2.6	Standardisierungsprozess und Governance	30
12.3	Ausblick	30
12.3.1	Kurzfristiger Horizont (2026–2028): Fundament legen	31
12.3.2	Mittelfristiger Horizont (2028–2031): Normative Verankerung	31
12.3.3	Langfristiger Horizont (2031–2035): Python als Erstsprache in AU- TOSAR	32
12.3.4	Die Rolle der internationalen Normungsgemeinschaft	34
12.3.5	Ein Appell an die Forschungsgemeinschaft	34
12.4	Schlussfolgerung	35
Literaturverzeichnis		35

1 Einleitung

Die Automobilindustrie steht an einem tiefgreifenden softwaretechnischen Wendepunkt. Software-defined Vehicles, zunehmende Fahrzeugelektronik, [Advanced Driver Assistance Systems \(ADAS\)](#)-Funktionen, Over-the-Air-Updates und KI-basierte Regelkreise erfordern eine drastische Steigerung der Softwareproduktivität sowie eine breitere Entwicklerbasis. Innerhalb des [AUTomotive Open System ARchitecture \(AUTOSAR\)](#)-Konsortiums wird derzeit intensiv diskutiert, die Programmiersprache Rust als offiziellen Bestandteil der Plattformspezifikationen zu verankern [5, 44].

Diese Arbeit argumentiert auf Basis technischer, ökonomischer, ökosystemischer und sicherheitstechnischer Analyse, dass *Python* – insbesondere in der Ausprägung *MicroPython* [15] – die strategisch überlegene Wahl für eine AUTOSAR-Standardisierung darstellt. Python bietet die weltgrößte Entwicklercommunity, konkurrenzlose Ökosystemreife, erstklassige KI/ML-Integration, ausgezeichnete Testbarkeit und etablierte Einsatznachweise in sicherheitskritischen Systemen. MicroPython ermöglicht dabei den direkten Einsatz auf ressourcenbeschränkten Mikrocontrollern, wie sie in der [AUTOSAR-Classic-Plattform](#) typisch sind, während CPython in der Adaptive Plattform nahtlos integrierbar ist.

Wir entwerfen eine vollständige Standardisierungsarchitektur, definieren Sprachprofilstufen (PySafety Level 1–4), schlagen einen Migrationspfad vor und diskutieren offene Forschungsfragen. Das Ergebnis ist ein konkreter, umsetzbarer Fahrplan für die Aufnahme von Python/MicroPython als normierte Sprache in AUTOSAR R24 und darüber hinaus.

Die Geschichte der Automobilelektronik ist eine kontinuierliche Eskalation der Softwarekomplexität. Während ein Fahrzeug der 1980er-Jahre kaum Software enthielt, umfassen moderne Fahrzeuge über 100 [Electronic Control Units \(ECUs\)](#), mehr als 100 Millionen Zeilen Quellcode und ein Dutzend unterschiedliche Kommunikationsbusse [46, 47]. Der Übergang zum Software-defined Vehicle (SDV) intensiviert diesen Trend weiter: Fahrzeugfunktionen sollen künftig primär durch Software definiert, aktualisiert und monetarisiert werden [48, 49].

Vor diesem Hintergrund gewinnt die Frage der Programmiersprachen-Standardisierung innerhalb des [AUTOSAR](#)-Rahmens strategische Bedeutung. [AUTOSAR](#) – gegründet 2003 von BMW, Bosch, Continental, DaimlerChrysler, Siemens VDO und Volkswagen – definiert eine offene, standardisierte Softwarearchitektur für Steuergeräte im Automobil [1]. Die Classic Plattform ([Classic Platform \(CP\)](#)) zielt auf Mikrocontroller mit begrenzten Ressourcen; die 2017 eingeführte Adaptive Plattform ([Adaptive Platform \(AP\)](#)) adressiert leistungsfähige Hochleistungsrechner mit POSIX-kompatiblen Betriebssystem [3].

Beide Plattformen spezifizieren bislang C (für CP) bzw. C++14/17 (für AP) als primäre Implementierungssprachen. Angesichts bekannter Schwächen dieser Sprachen bezüglich Speichersicherheit [53] formierte sich eine Gemeinschaft, die Rust als modernere, speichersichere Alternative in AUTOSAR einzubringen sucht [5, 6]. Rust besitzt zweifellos Vorzüge in Bezug auf Speichersicherheit durch sein Ownership-Modell [43], jedoch bringt es erhebliche Hindernisse mit sich: steile Lernkurve, kleine Entwicklercommunity im Automotive-Bereich, unreifes Tooling und fehlende Normnachweise für sicherheitskritische Anwendungen.

Diese Arbeit positioniert Python/MicroPython als die überlegene Alternative. Während Rust eine Nische Systemprogrammiersprache bleibt, ist Python die meistgenutzte Programmiersprache der Welt [64, 65] mit einem Ökosystem, das Machine Learning, formale Verifikation, Testautomatisierung, Protokoll-Scripting und Hardware-nahe Programmierung gleichermaßen abdeckt. MicroPython [15, 16] – eine schlanke, auf

Mikrocontroller optimierte Python-3-Implementierung – beweist täglich in Millionen industrieller und akademischer Anwendungen, dass Python auch auf ressourcenbeschränkter Hardware praktikabel ist.

Die Forschungsfragen dieser Arbeit lauten:

1. Welche technischen Anforderungen stellt [AUTOSAR](#) an eine Programmiersprache, und inwiefern erfüllt Python/MicroPython diese?
2. Wie lässt sich ein Python-Sprachprofil für AUTOSAR sicherheitstechnisch spezifizieren, das ISO 26262 konform ist?
3. Welche Architektur ermöglicht die Integration von MicroPython in die AUTOSAR Classic Platform und Python in die Adaptive Platform?
4. Welche wirtschaftlichen und ökosystemischen Vorteile entstehen durch eine Python-Standardisierung?
5. Wie verhält sich Python/MicroPython im Vergleich zu Rust hinsichtlich der für AUTOSAR relevanten Kriterien?

Der Rest dieser Arbeit ist wie folgt strukturiert: Abschnitt [2](#) erläutert die [AUTOSAR](#)-Architektur im Detail. Abschnitt [3](#) stellt Python und MicroPython vor. Abschnitt [4](#) analysiert die Rust-Bewegung kritisch. Abschnitt [5](#) untersucht die technische Eignung. Abschnitt [6](#) behandelt Functional Safety und Cybersecurity. Abschnitt [7](#) entwirft die Integrationsarchitektur. Abschnitt [8](#) diskutiert das Ökosystem. Abschnitt [9](#) analysiert den wirtschaftlichen Nutzen. Abschnitt [10](#) formuliert den Standardisierungsvorschlag. Abschnitt [11](#) diskutiert verwandte Arbeiten. Abschnitt [12](#) schließt mit Fazit und Ausblick.

2 AUTOSAR – Architektur und Anforderungen

2.1 Geschichte und Motivation

[AUTOSAR](#) wurde 2003 als industrielles Konsortium gegründet [\[1\]](#), um die wachsende Fragmentierung der Steuergeräte-Software zu beenden. Vor AUTOSAR pflegte jeder OEM und jeder Tier-1-Zulieferer eigene, inkompatible Software-Stacks für jede Derivatefamilie. Code war nicht portierbar, Integrationsprojekte hochkostenintensiv, und Fehler von Zulieferern schwer zu isolieren.

AUTOSAR definierte eine layered architecture mit klar spezifizierten Schnittstellen: Microcontroller Abstraction Layer ([Microcontroller Abstraction Layer \(MCAL\)](#)), [Basic Software \(BSW\)](#), [Runtime Environment \(RTE\)](#) und [Anwendungs-Software Components \(SWCs\)](#) [\[4\]](#). Durch standardisierte [Application Programming Interfaces \(APIs\)](#) und eine XML-basierte Beschreibungssprache (ARXML) konnten Softwarekomponenten erstmals modular ausgetauscht werden.

2.2 AUTOSAR Classic Platform (CP)

Die Classic Platform zielt auf Mikrocontroller der Klasse:

- RAM: 32 kB bis 4 MB
- Flash: 256 kB bis 16 MB

- CPU: 8- bis 32-Bit, 8–800 MHz

CP spezifiziert C (ISO C99, ergänzt durch MISRA C [12]) als Implementierungssprache. Der AUTOSAR CP-Stack umfasst über 80 Module, darunter OS (nach OSEK [14]), [Controller Area Network \(CAN\)](#)-Stack, Diagnostics (UDS), Flash Management und mehr [2].

2.3 AUTOSAR Adaptive Platform (AP)

Die Adaptive Platform adressiert leistungsstarke Service-orientierte Architekturen auf Hochleistungs-SoCs (z.B. NXP S32G, Renesas R-Car) mit:

- Multi-Core ARM-A-Prozessoren, 1–16 Kerne, 1–3 GHz
- RAM: 1 GB bis 16 GB
- Ethernet/ SOME/IP als primärem Kommunikationsbus
- POSIX-Betriebssystem (z.B. Linux, QNX, Integrity)

AP nutzt C++14/17 als Sprache und definiert die AUTOSAR Runtime for Adaptive Applications ([AUTOSAR Runtime for Adaptive Applications \(ARA\)](#)) [3]. Dienste (Service-oriented Communication via SOME/IP) ersetzen weitgehend das Signal-orientierte Paradigma von CP.

2.4 Anforderungskatalog an eine Programmiersprache in AUTOSAR

Aus den AUTOSAR-Spezifikationen und den einschlägigen Normen ISO 26262 [10] und ISO 21434 [11] lassen sich folgende Anforderungen an eine Programmiersprache ableiten:

Tabelle 1: Anforderungskatalog für Programmiersprachen in AUTOSAR

ID	Anforderung	Quelle
R1	Deterministisches Laufzeitverhalten; keine unvorhersehbaren Latenzspitzen	ISO 26262
R2	Qualifizierter Compiler / Interpreter (Tool Confidence Level)	ISO 26262-8
R3	Statische Analyse möglich (Typprüfung, Kontrollfluss)	MISRA C
R4	Definiertes Speichermodell; keine unkontrollierten Heap-Operationen	ISO 26262
R5	Sprachsubset definierbar für ASIL-B/C/D-Anwendungen	ISO 26262-6
R6	Formale oder halbformale Verifikation unterstützt	IEC 61508
R7	Breite Toolchain-Unterstützung (Debugger, Profiler)	Praxis
R8	Hardware-nahe Programmierung möglich (Register, DMA, Interrupts)	CP-Anforderung

(Fortsetzung von vorheriger Seite)

ID	Anforderung	Quelle
R9	Interoperabilität mit C/C++ (FFI)	Migration
R10	Ausreichend große, in Automotive erfahrene Entwicklercommunity	OEM-Anforderung
R11	Offene Lizenzierung (keine proprietären Lock-ins)	AUTOSAR-Statut
R12	Laufzeitfehler-Erkennung (Exceptions, Assertions)	ISO 26262-6
R13	Deterministische Speicherbereinigung kontrollierbar	Embedded
R14	Cybersecurity-relevante Sprachkonstrukte (keine Buffer Overflows)	ISO 21434
R15	Testbarkeit (Unit, Integration, Mutations-test)	ISO 26262-6

3 Python und MicroPython – Überblick und Eigenschaften

3.1 Python – Die universelle Hochsprache

Python wurde 1991 von Guido van Rossum veröffentlicht [24] und hat sich zur meistgenutzten Programmiersprache der Welt entwickelt [64]. Die Sprache vereint Prinzipien von Lesbarkeit, Ausdrucksstärke und Produktivität. Charakteristisch sind:

- **Dynamisches Typsystem** mit optionalem statischem Typing (seit PEP 484, Python 3.5) durch Typ-Annotationen und das `typing`-Modul [25];
- **Garbage Collected Memory Management** mit Referenzzählung als primärem Mechanismus (deterministischer als Mark-and-Sweep für kurzlebige Objekte);
- **Klare, erzwungene Syntaxstruktur** durch einrückungsbasierte Blöcke;
- **Erstklassige Funktionen, Closures, Generatoren** und `async/await`-Coroutinen (seit Python 3.5);
- **Umfangreiches Standardbibliotheks-Ökosystem** (über 200 000 Pakete auf PyPI [26]);
- **C-Erweiterbarkeit** via CPython C-API oder Cython [40];
- **Vollständige Unicode-Unterstützung** (PEP 3120).

3.1.1 Statische Typprüfung und Verifikation

Ein weitverbreiteter Kritikpunkt an Python ist das dynamische Typsystem. Dieser ist durch moderne Werkzeuge weitgehend obsolet: `mypy` [29], `pyright` [30], `pytype` (Google)

und **pyre** (Meta) bieten vollständige statische Typprüfung. Kombiniert mit strikten Typ-Annotationen (PEP 484, PEP 526, PEP 544) lässt sich Python-Code auf einem Niveau typprüfen, das C++ mit modernen Compilern vergleichbar ist [34].

Für sicherheitskritische Anwendungen ist dies entscheidend: Ein AUTOSAR-SWC könnte mit vollständiger Typ-Annotation versehen werden, und die statische Analyse via mypy/pyright würde Typfehler bereits vor dem Lauf entdecken, analog zu einem C-Compiler mit Warnungen.

3.1.2 CPython und alternative Implementierungen

Neben der Referenzimplementierung CPython existieren:

- **MicroPython:** Für Mikrocontroller (s. Abschnitt 3.2);
- **PyPy:** JIT-kompilierter Interpreter, bis zu $5\times$ schneller als CPython [37];
- **Nuitka** [38]: Kompiliert Python zu nativem C-Code;
- **Cython:** Superset von Python, kompiliert zu C [40];
- **mypyc:** Kompiliert typierten Python-Code zu C-Erweiterungen [39].

Insbesondere **Nuitka** und **mypyc** sind für die AUTOSAR-Adaptive-Plattform hochrelevant: Sie ermöglichen die Erstellung von nativen ELF-Binaries aus Python-Quellcode, die direkt in POSIX-ECUs deploybar sind, ohne einen Interpreter im Produktionssystem laufen zu lassen.

3.2 MicroPython – Python für Mikrocontroller

MicroPython wurde 2013–2014 von Damien P. George entwickelt und durch eine Kickstarter-Kampagne finanziert [16]. Es ist eine vollständige, schlanke Python-3-Implementierung speziell für ressourcenbeschränkte Hardware [15]:

- **Minimaler Flash-Bedarf:** ca. 256 kB (Basis-Interpreter); typische Deployment-Größe 256–512 kB Flash;
- **Minimaler RAM-Bedarf:** lauffähig ab 16 kB RAM, typisch 64–256 kB;
- **Python-3-Kompatibilität:** Kern-Sprachfeatures vollständig (Klassen, Generatoren, Closures, try/except, async/await);
- **Hardware-APIs:** `machine.Pin`, `machine.UART`, `machine.SPI`, `machine.I2C`, `machine.Timer`, `machine.ADC`, `machine.WDT` u.v.m.;
- **Inline-Assembler** für ARM Thumb2, Xtensa und STM32;
- **Native Code Emitter:** `@micropython.native`-Decorator kompiliert zu native Maschinencode ($4\text{--}10\times$ schneller);
- **Viper-Code-Emitter:** `@micropython.viper`-Decorator mit C-ähnlichem Typsystem für höchste Performance;
- **Integrierter Garbage Collector** (Mark-and-Sweep mit konfigurierbarer Heap-Größe und manuell triggerbarer Kollektion).

3.2.1 Unterstützte Mikrocontroller-Plattformen

MicroPython unterstützt eine breite Plattformpalette [17]:

Tabelle 2: Von MicroPython offiziell unterstützte Plattformen (Auswahl)

Port	Architektur	Typische Automotive-MCU	Status
stm32	ARM Cortex-M	STM32F4/F7/H7, Infineon TC3xx	Stabil
esp32	Xtensa LX7, RISC-V	ESP32-S3, C6	Stabil
rp2	ARM Cortex-M0+	RP2040, RP2350	Stabil
nrf	ARM Cortex-M	nRF52840	Stabil
renesas-ra	ARM Cortex-M	RA4/RA6	Beta
samd	ARM Cortex-M0+	SAMD51	Stabil
unix	x86/ARM64	Linux-ECU (AP)	Stabil
windows	x86_64	Simulation/HIL	Stabil

3.2.2 MicroPython Performance

Die Performance-Eigenschaften von MicroPython sind für eingebettete Systeme gut untersucht [20, 23]:

- Standardmäßiger bytecode-interpretierter Python-Code: ca. 10–50× langsamer als äquivalenter C-Code;
- Mit `@micropython.native`: ca. 2–5× langsamer als C;
- Mit `@micropython.viper`: annähernd C-Geschwindigkeit (1–2×);
- Inline-Assembler: gleichwertig zu handoptimiertem Assembler.

Dies ist für nicht-zeitkritische Schichten von AUTOSAR-Software (Diagnose, Konfiguration, OTA-Management, Kommunikationsprotokoll-Glue-Code) vollkommen ausreichend und für zeitkritische Schichten durch native/viper-Dekoratoren oder C-Extensions lösbar.

3.2.3 MicroPython Speicherverwaltung im Detail

Das Speicherverwaltungsmodell von MicroPython ist für Embedded Systems ausgelegt:

- **Heap-Allokation** nur aus einem konfigurierbaren, statisch reservierten Speicherbereich (kein malloc/free aus dem OS-Heap);
- **Deterministisches GC**: GC kann deaktiviert und manuell über `gc.collect()` ausgelöst werden;
- **Worst-Case Execution Time (WCET)**: Mit deaktiviertem GC und viper-Decorator ist WCET analysierbar;
- **Pre-Allokation**: Kritische Objekte können zur Boot-Zeit pre-allokiert und aus dem GC ausgeschlossen werden (`micropython.const`);

- **Frozen Modules:** Python-Module können als Bytecode in Flash eingefroren werden (kein RAM für Quellcode nötig).

```

1 import gc
2 import micropython
3
4 # Pre-allokiere kritische Puffer zur Boot-Zeit
5 _can_rx_buffer = bytearray(64)
6 _can_tx_buffer = bytearray(64)
7
8 # Deaktiviere GC fuer zeitkritische Interrupt-Handler
9 @micropython.native
10 def can_isr_handler(frame_id: int, data: memoryview) -> None:
11     """CAN-Interrupt-Handler: GC-frei, deterministisch."""
12     gc.disable() # Kein GC waehrend ISR
13     try:
14         _can_rx_buffer[:len(data)] = data
15         _process_frame_native(frame_id, _can_rx_buffer)
16     finally:
17         gc.enable()
18
19 @micropython.viper
20 def _process_frame_native(frame_id: int, buf: ptr8) -> int:
21     """Viper-Code: annaehernd C-Geschwindigkeit."""
22     if frame_id == 0x100:
23         return int(buf[0]) << 8 | int(buf[1])
24     return -1

```

Listing 1: MicroPython Speichermanagement für zeitkritische Funktionen

4 Analyse der Rust-Standardisierungsbewegung in AUTOSAR

4.1 Die Rust-Bewegung im Überblick

Rust [43] wurde von Mozilla Research entwickelt und 2015 in Version 1.0 veröffentlicht. Seine Haupteigenschaft ist das Ownership-System, das Speichersicherheit ohne Garbage Collector zur Compilezeit garantiert. Dies macht Rust attraktiv für Systemprogrammierung.

Im Automobilbereich wurde eine AUTOSAR-Arbeitsgruppe für Rust gebildet [6], und das Whitepaper “Safe Rust Guidelines for Automotive [5]” beschreibt erste Richtlinien. Die MISRA-Community diskutiert MISRA Rust [13].

4.2 Kritische Analyse der Rust-Eignung für AUTOSAR

Trotz dieser Aktivitäten bestehen fundamentale Hindernisse:

4.2.1 Lernkurve und Entwicklerengpass

Rust gilt als eine der schwierigsten Programmiersprachen [65]. Das Ownership-/Borrow-Modell ist konzeptuell neuartig und erfordert erhebliche Einarbeitungszeit. Während Python nach TIOBE [64] und dem Stack Overflow Survey [65] weltweit die meistgenutzte Sprache ist (ca. 30 Mio. aktive Entwickler), zählt Rust nur ca. 3 Mio. Entwickler weltweit

- davon ein Bruchteil mit Automotive-Erfahrung. Der resultierende Entwicklermangel verteuert Projekte und verlängert Time-to-Market erheblich.

4.2.2 Fehlendes Safety-Zertifizierungs-Ökosystem

ISO 26262-8 verlangt für Sprachtools einen *Tool Confidence Level* [10]. Für Rust existiert (Stand 2025) kein zertifizierter Compiler (nach ISO 26262 ASIL-D qualifiziert), während GCC und LLVM/Clang seit Jahren entsprechend qualifiziert werden (z.B. durch AbInt, AdaCore, ETAS). MicroPython und CPython können auf qualifizierte C-Backends (LLVM/Clang, GCC mit Qualifizierungskit) aufsetzen.

4.2.3 Fehlende AUTOSAR-Treiberintegration

Automotive-spezifische HAL-Bibliotheken (CAN, LIN, FlexRay, Ethernet, diagnostischer Speicherzugriff) existieren für C über Jahrzehnte. Für Rust sind automotive-grade BSW-Bindungen weitgehend nicht existent und müssten von Grund auf neu entwickelt werden – ein Jahrzehnt-Projekt für die gesamte Industrie.

4.2.4 ‘unsafe’ Rust: kein Gewinn gegenüber C

Viele Hardware-nahe Operationen in Automotive-ECUs erfordern zwingend `unsafe`-Rust-Blöcke [45]. Damit sind die Speichersicherheits-Garantien von Rust in den kritischsten Schichten (MCAL, Treiber) ohnehin nicht gegeben. Python/MicroPython mit seiner klar definierten Sicherheitsschicht (C-Extension in sicherem C) ist hier transparent.

4.2.5 Zusammenfassung: Rust vs. Python/MicroPython

5 Technische Eignung von Python/MicroPython für AUTOSAR

5.1 Erfüllung des Anforderungskatalogs

Wir prüfen systematisch, inwieweit Python/MicroPython die in Tabelle 1 definierten Anforderungen erfüllt:

R1 – Deterministisches Laufzeitverhalten MicroPython erlaubt die Kontrolle des GC, die Verwendung von `@micropython.native` und `@micropython.viper` sowie Inline-Assembler für zeitkritische Pfade. Durch Pre-Allokation kritischer Objekte und manuelle GC-Steuerung (vgl. Listing 1) ist deterministisches Verhalten erzielbar. Für ASIL-D-Funktionen wäre eine static-alloc-only-Bibliothek analog zu `heapless` in Rust zu spezifizieren.

R2 – Qualifizierter Interpreter CPython kompiliert zu C-Code, der mit qualifizierten GCC/Clang-Compilern übersetzt wird. MicroPython wird ebenfalls mit GCC/Clang kompiliert. Eine Qualifizierung des MicroPython-Interpreters selbst nach Tool Confidence Level (TCL2) ist machbar – vergleichbar mit der Qualifizierung von Java VM-Implementierungen nach IEC 61508 [58].

Tabelle 3: Vergleich Rust vs. Python/MicroPython für AUTOSAR-Standardisierung

Kriterium	Rust	Python/MicroPython
Entwicklerbasis weltweit	~3 Mio.	>30 Mio.
Automotive-Erfahrung (Community)	sehr gering	mittel–hoch (Skripte, Tests, Tools)
Lernkurve	sehr steil	flach
Compiler-Qualifizierung (ISO 26262)	nicht vorhanden	via GCC/LLVM möglich
AUTOSAR BSW-Bindungen	keine	C-FFI, erste Bibliotheken
MISRA-äquivalentes Profil	MISRA Rust (Draft)	PyMISRA-Ansätze (forschbar)
Embedded-Footprint	50–500 kB (no_std)	256 kB (MicroPython)
KI/ML-Integration	rudimentär	erstklassig (PyTorch, TensorFlow)
OTA-Scripting	nicht vorgesehen	nativ via Python-Scripts
Testökosystem	wachsend	ausgereift (pytest, hypothesis)
Statische Analyse	sehr gut	sehr gut (mypy, pyright)
Formale Verifikation	experimentell	SPARK-ähnlich via typsichere Teilmengen
Hardware-nahe Ops	native (unsafe)	viper-Decorator, Inline-ASM
Industrielle Nachweise	kaum	MicroPython in Industrie weit verbreitet

R3 – Statische Analyse mypy mit strikter Konfiguration (`-strict`) und pyright ermöglichen vollständige statische Typprüfung. Zusätzlich bieten `pylint` [31], `flake8`, `bandit` (Sicherheitsanalyse) und `pyflakes` umfangreiche Codeanalyse. Für AUTOSAR-spezifische Regeln wäre ein *AUTOSAR Python Checker* als Open-Source-Tool definierbar.

R4 – Definiertes Speichermodell MicroPython: gesamter Heap ist ein konfigurierbarer, statischer Buffer. Keine Interaktion mit OS-Heap. Vollständig kontrollierbar. Für Adaptive Platform: Python mit speziell allokierten arenas (CPython konfigurierbar via `pymalloc` oder `custom allocator` [41]).

R5 – Sprachsubset für ASIL Python besitzt eine klar definierte, kleine Kernsprache. Ein ASIL-Sprachsubset (“PySafety-Profile”) ist eindeutig definierbar: Verboten wären z.B. `eval()`, `exec()`, dynamischer Import, und unbeschränkte Rekursion. Erlaubt wären typisierte Funktionen, Pre-allokierte Datenstrukturen und native-Dekoratoren. (s. Abschnitt 6.1.2)

R6 – Formale Verifikation Für typisierte Python-Teilmengen existieren Ansätze zu Model Checking (via Abstraktion auf Typsystem-Ebene) [34]. Das Projekt *nagini* [33]

implementiert Verifikation von Python-Code via Viper-Assertion-Sprache.

R7 – Toolchain Python verfügt über ausgezeichnete Toolchain-Unterstützung: Eclipse/VS Code-Integration, REPL für interaktives Debugging, GDB-Integration via `python-gdb.py` [42], Profiler (`cProfile`, `line_profiler`), Memory-Analyzer (`tracemalloc`).

R8 – Hardware-nahe Programmierung MicroPython bietet native Hardware-APIs. Listing 2 zeigt exemplarisch den direkten Registerzugriff auf einem Cortex-M-Controller.

```
1 import micropython
2 from micropython import const
3
4 # Direkte Speicher-mapped Register-Adresse (Cortex-M)
5 _RCC_BASE = const(0x40023800)
6 _RCC_AHB1ENR = const(_RCC_BASE + 0x30)
7 _GPIOA_MODER = const(0x40020000)
8
9 @micropython.viper
10 def enable_gpioa_clock() -> None:
11     """Aktiviere GPIOA-Takt ueber RCC-Register (direkte Schreiboperation)"""
12     # Zeiger auf uint32-Register
13     rcc_ahb1enr = ptr32(_RCC_AHB1ENR)
14     rcc_ahb1enr[0] = rcc_ahb1enr[0] | 0x1 # Bit 0: GPIOAEN
15
16 @micropython.viper
17 def config_pin_output(pin: int) -> None:
18     """Konfiguriere GPIO-Pin als Push-Pull-Ausgang."""
19     moder = ptr32(_GPIOA_MODER)
20     moder[0] = (moder[0] & ~(0x3 << (pin * 2))) | (0x1 << (pin * 2))
```

Listing 2: Direkter Hardware-Registerzugriff mit MicroPython

R9 – Interoperabilität mit C/C++ MicroPython unterstützt native C-Module (.c-Dateien, die als MicroPython-Module kompiliert werden) [18]. CPython bietet die vollständige C-API sowie `ctypes`, `cffi` und `Cython`. Bestehende AUTOSAR-BSW-C-Bibliotheken können direkt von Python aus aufgerufen werden.

R10–R15 Alle weiteren Anforderungen werden in den folgenden Abschnitten (Sicherheit, Architektur, Ökosystem) detailliert behandelt.

5.2 MicroPython auf Automotive-Mikrocontrollern

Automotive-Mikrocontroller der Generation 2020+ haben typische Spezifikationen, die MicroPython komfortabel unterstützen:

Die Ressourcen moderner Automotive-MCUs übertreffen die MicroPython-Mindestanforderungen um einen Faktor von 16–64× (Flash) bzw. 8–128× (RAM). Selbst ohne die gesamte Python-Stdlib ist genügend Platz für eine vollwertige MicroPython-Laufzeit plus applikationsspezifischen Code.

Tabelle 4: Ressourcenvergleich: MicroPython-Anforderungen vs. Automotive-MCUs

Ressource	MicroPython Min.	Renesas RH850/F1x	Infineon TC39x
Flash	256 kB	4 MB	16 MB
RAM	64 kB	512 kB	8 MB
CPU-Takt	8 MHz	240 MHz	300 MHz
CPU-Architektur	ARM/RISC-V	V850 / ARM	TriCore

5.3 Latenz und Interrupt-Handling

Interrupt-Service-Routinen ([Interrupt-Service-Routines \(ISRs\)](#)) sind der hardwarenächste Teil des Systems. MicroPython bietet mehrere Möglichkeiten:

1. **C-ISR:** Interrupt-Handler in C implementiert, ruft Python-Callback via `micropython.schedule()` aus sicherem Kontext auf;
2. **Native ISR:** `@micropython.native`-dekorierte Python-Handler, direkt als ISR registrierbar (`machine.Pin.irq(..., handler=fn)`);
3. **Deferred Execution:** ISR setzt Flag, Hauptschleife verarbeitet asynchron (klassisches Muster in embedded systems).

Messungen auf STM32F4 (168 MHz Cortex-M4) zeigen [21]:

- C-ISR Latenz: $<1 \mu\text{s}$
- Native Python-ISR Latenz: $2\text{--}5 \mu\text{s}$
- Deferred Python-Callback: $10\text{--}50 \mu\text{s}$ (abhängig von Scheduling)

Für AUTOSAR-CP-Anwendungen mit typischen Zykluszeiten von 1 ms (Task Class 1) bis 100 ms (Task Class 3) sind diese Latenzen vollkommen akzeptabel. Nur harte Echtzeit-ISRs ($<1 \mu\text{s}$) würden weiterhin native C erfordern.

6 Sicherheitstechnische Betrachtung

6.1 Functional Safety nach ISO 26262

ISO 26262 [10] definiert vier Automotive Safety Integrity Levels (ASIL A bis ASIL D, aufsteigend kritisch). Die Auswahl der Programmiersprache und ihrer Laufzeitumgebung fließt in den Safety Case ein.

6.1.1 Anforderungen an die Sprachlaufzeit

ISO 26262-6 (Software-Level) fordert:

- Verwendung sprach-spezifischer Sprachguidelines (Subset, Coding Guidelines);
- Nachweis der Werkzeugqualifizierung;
- Statische Analyse, Kontrollflussanalyse, Datenflussanalyse;
- Dynamisches Testen (Statement Coverage ASIL-A/B, Branch Coverage ASIL-C, MC/DC ASIL-D).

6.1.2 PySafety-Sprachprofil

Wir definieren vier **PySafety-Level** als AUTOSAR-Erweiterung:

Definition 6.1 (PySafety Level). *Ein PySafety Level beschreibt ein standardisiertes Sprachsubset von Python/MicroPython, das für eine bestimmte ASIL-Klasse einsetzbar ist.*

Tabelle 5: PySafety-Level-Definitionen

Level	Erlaubte Sprachfeatures	Verbotene Features	ASIL-Mapping
PSL-0	Vollständiges Python	–	QM
PSL-1	Python ohne eval/exec, ohne dynamischen Import	eval, exec, <code>__import__</code>	ASIL-A
PSL-2	PSL-1 + vollständige Typ-Annotationen, <code>*args/**kwargs</code> ohne Typen	Dynamische Attribute, getattr ohne Typcheck	ASIL-B
PSL-3	PSL-2 + nur pre-allokierte Container, kein unbeschränkter Heap	<code>list()</code> , <code>dict()</code> ohne Größenbeschränkung, offene Rekursion	ASIL-C
PSL-4	PSL-3 + nur viper/native-Dekoratoren in ISR-Pfaden, belegter WCET-Nachweis	Alle GC-triggernden Operationen in zeitkritischen Pfaden	ASIL-D

6.1.3 Coding Guidelines und statische Prüfung

Analog zu MISRA C [12] wären **AUTOSAR Python Guidelines** zu definieren. Erste Entwürfe könnten auf bestehenden Arbeiten aufbauen:

- **PySafety Coding Rules:** Ein Satz von ca. 80–120 Regeln, die mit `pylint`-Plugin oder eigenem Checker automatisiert prüfbar sind;
- **NASA PySafe:** NASA’s Python-Codierungsrichtlinien für sicherheitskritische Systeme [55];
- **CERT Secure Coding Standard für Python** [54].

6.2 Cybersecurity nach ISO 21434

ISO 21434 [11] regelt Cybersecurity im Fahrzeugbereich. Python bietet hier signifikante Vorteile gegenüber C/C++:

- **Keine Buffer Overflows:** Python’s Memory-Model verhindert klassische Stack- und Heap-Overflows on language level;
- **Keine Format-String-Vulnerabilities:** f-Strings und `str.format()` sind typischer;
- **Integer-Overflow frei:** Python’s `int` ist beliebig groß (in MicroPython: 30-bit integer intern, erweiterbar);

- **Kryptographie-Standardbibliothek:** `hashlib`, `hmac`, `ssl` (CPython) oder `ucryptolib` (MicroPython);
- **Security Linter:** `bandit` [32] scannt Python-Code auf OWASP-Top-10-Schwachstellen.

Für Automotive gelten besondere Anforderungen:

- UDS-Sicherheits-Services (Session-Management, Seed-Key-Algorithmus) in Python implementierbar mit kryptografischen Primitiven;
- SecureBoot-Verifikation via `ucryptolib.aes` auf MicroPython;
- TLS 1.3-Kommunikation via `ussl` auf MicroPython oder `mbedtls`-Binding.

```

1  import ucryptolib
2  import hashlib
3  import ubinascii
4
5  # AES-128-CBC fuer SecureFlash-Verifikation
6  def verify_firmware_signature(
7      firmware: bytes,
8      expected_hmac: bytes,
9      key: bytes
10 ) -> bool:
11     """Pruefe HMAC-SHA256 einer Firmware (MicroPython)."""
12     import hmac
13     computed = hmac.new(key, firmware, hashlib.sha256).digest()
14     # Constant-time Vergleich (timing-attack-safe)
15     return _constant_time_compare(computed, expected_hmac)
16
17 def _constant_time_compare(a: bytes, b: bytes) -> bool:
18     """Zeitkonstanter Byte-String-Vergleich."""
19     if len(a) != len(b):
20         return False
21     result = 0
22     for x, y in zip(a, b):
23         result |= x ^ y
24     return result == 0

```

Listing 3: Sicherheits-relevante Kryptographie in MicroPython

6.3 WCET-Analyse und Echtzeitfähigkeit

Worst-Case Execution Time Analysis (WCET) ist für ASIL-C/D-Funktionen zwingend erforderlich [57]. Für Python/MicroPython-Code ist WCET auf drei Ebenen möglich:

1. **Viper-Level:** Code mit `@micropython.viper` ist strukturell äquivalent zu C-Code und kann mit etablierten WCET-Werkzeugen (AbsInt aiT [56], TriCore WCET-Analyzer) analysiert werden – der Viper-Code wird zu identischem Maschinencode wie äquivalentes C kompiliert;
2. **Bytecode-Level:** Für bytecode-interpretierten Python-Code ist WCET durch Analyse des Interpreters + Bytecode berechenbar (Forschungsansatz: [22]);
3. **Partitionierung:** Zeitkritische Funktionen in C, zeitunkritische in Python – bewährter Praxis-Ansatz.

7 Integrationsarchitektur

7.1 Überblick: Python in der AUTOSAR-Schichtenarchitektur

AUTOSAR Application Layer (SWCs)
Python / MicroPython SWC Layer
(Python-SWCs, typisiert, PSL-1..4)
PySafety Runtime (PyAutosar-RTE)
(Python-RTE-Schnittstelle, C-FFI-Bridge)
AUTOSAR RTE (Standard C)
AUTOSAR BSW (Standard C)
(COM, NM, DEM, DCM, NvM, ...)
MCAL (C, Hersteller-spezifisch)
Hardware (ECU, MCU, Peripherie)

Abbildung 1: Integrierte AUTOSAR-Schichtenarchitektur mit Python/MicroPython SWC-Ebene

7.2 Python SWC Integration – Classic Platform

Für die AUTOSAR Classic Platform wird folgende Integrationsarchitektur vorgeschlagen:

7.2.1 PyAutosar-RTE-Bridge

Eine C-Schicht (`pyautosar_rte.c`) bildet die AUTOSAR-RTE-API auf Python-callable C-Funktionen ab:

```
1  /* pyautosar_rte.c: Bridge zwischen AUTOSAR RTE und MicroPython VM */
2  #include "Rte_Type.h"
3  #include "Rte_MySwc.h"
4  #include "py/runtime.h" /* MicroPython C-API */
5
6  /* Python-callable: Rte_Write_Port_Signal */
7  STATIC mp_obj_t mp_Rte_Write_VehicleSpeed(mp_obj_t speed_obj) {
8      float speed = mp_obj_get_float(speed_obj);
9      Std_ReturnType ret = Rte_Write_MySwc_VehicleSpeed(
10         (Float32)speed
11     );
12     return mp_obj_new_int(ret);
13 }
14 STATIC MP_DEFINE_CONST_FUN_OBJ_1(
15     mp_Rte_Write_VehicleSpeed_obj,
16     mp_Rte_Write_VehicleSpeed
17 );
18
19 /* Modul-Registrierung */
20 STATIC const mp_rom_map_elem_t autosar_rte_module_globals_table[] = {
21     { MP_ROM_QSTR(MP_QSTR__name__), MP_ROM_QSTR(MP_QSTR_autosar_rte) },
22     { MP_ROM_QSTR(MP_QSTR_write_vehicle_speed),
23         MP_ROM_PTR(&mp_Rte_Write_VehicleSpeed_obj) },
24 };
```

Listing 4: PyAutosar-RTE-Bridge: C-Seite (Auszug)

```

1  """
2  Beispiel: Python-implementierter AUTOSAR SWC fuer
3  Fahrzeuggeschwindigkeit
4  PSL-3 compliant: Typ-Annotationen, keine dynamischen Allokierungen
5  """
6  from autosar_rte import write_vehicle_speed, read_wheel_speeds
7  import micropython
8
9  # Kalibrierungsdaten: pre-allokiert, const
10 WHEEL_CIRC_MM: float = micropython.const(1975.0) # 215/55R17
11 PULSES_PER_REV: int = micropython.const(48)
12
13 @micropython.native
14 def runnable_calc_speed() -> None:
15     """
16     AUTOSAR Runnable: 10ms zyklisch aufgerufen.
17     Liest Raddrehzahl-Pulse, berechnet Fahrzeuggeschwindigkeit.
18     """
19     pulse_fl: int
20     pulse_fr: int
21     pulse_rl: int
22     pulse_rr: int
23     pulse_fl, pulse_fr, pulse_rl, pulse_rr = read_wheel_speeds()
24
25     # Durchschnitt Hinterachse (Antriebsachse)
26     avg_pulses: int = (pulse_rl + pulse_rr) >> 1
27
28     # Umrechnung: Pulse/10ms -> km/h
29     #  $v \text{ [km/h]} = (\text{pulses} * \text{circ\_mm}) / (\text{pulses\_per\_rev} * 10\text{ms}) * 3.6$ 
30     speed_kmh: float = (
31         avg_pulses * WHEEL_CIRC_MM * 3.6
32     ) / (PULSES_PER_REV * 10.0)
33
34     write_vehicle_speed(speed_kmh)

```

Listing 5: Python SWC unter AUTOSAR CP

7.2.2 MicroPython-VM Integration in AUTOSAR-OS

Die MicroPython-Virtuelle Maschine (VM) wird als eigenständiger AUTOSAR-Task unter dem AUTOSAR-OS (OSEK-basiert) gestartet:

```

1  /* autosar_python_task.c */
2  #include "Os.h"
3  #include "py/compile.h"
4  #include "py/runtime.h"
5  #include "py/gc.h"
6
7  /* Statisch allokiertem Heap fuer MicroPython VM */
8  static uint8_t mp_heap[32768]; /* 32 kB Python-Heap */
9
10 TASK(PythonRunnableTask) {
11     /* Initialisierung (einmalig beim ersten Aufruf) */
12     static bool initialized = false;
13     if (!initialized) {
14         gc_init(mp_heap, mp_heap + sizeof(mp_heap));
15         mp_init();

```

```

16     initialized = true;
17     /* Lade Python-SWC aus gefrorenem Modul */
18     mp_import_mod(MP_QSTR_swc_speed_calc);
19 }
20 /* Rufe Python-Runnable auf */
21 mp_call_function_0(
22     mp_load_global(MP_QSTR_runnable_calc_speed)
23 );
24 TerminateTask();
25 }

```

Listing 6: MicroPython VM als AUTOSAR-OS-Task

7.3 Python Integration – Adaptive Platform

Auf der Adaptive Platform ist die Integration trivial: CPython (oder Nuitka-kompiliertes Python) läuft als nativer POSIX-Prozess. Die ara-Schnittstellen ([ARA](#)) werden als Python-Bindings bereitgestellt:

```

1  """
2  AUTOSAR Adaptive Platform: Python Service Application.
3  Kommuniziert via SOME/IP ueber ara::com Python-Bindings.
4  """
5  import asyncio
6  from ara.com import ServiceProxy, ServiceSkeleton
7  from ara.log import Logger
8  from ara.exec import ApplicationClient
9  from typing import Optional
10
11  logger = Logger.create_logger("SpeedService")
12
13  class VehicleSpeedSkeleton(ServiceSkeleton):
14      """AUTOSAR AP Service-Skeleton in Python."""
15
16      async def get_speed(self) -> float:
17          """Gibt aktuelle Fahrzeuggeschwindigkeit zurueck (km/h)."""
18          return self._current_speed
19
20      async def on_speed_update(self, speed: float) -> None:
21          """Event-Handler: Neue Geschwindigkeit empfangen."""
22          logger.info(f"Geschwindigkeitsupdate: {speed:.1f} km/h")
23          self._current_speed = speed
24          await self.notify_speed_changed(speed)
25
26      async def main() -> None:
27          """AUTOSAR AP Application Main-Coroutine."""
28          app = ApplicationClient("SpeedService")
29          skeleton = VehicleSpeedSkeleton(
30              instance_id="SpeedService/1"
31          )
32
33          async with app.run():
34              await skeleton.offer_service()
35              logger.info("SpeedService angeboten -- warte auf Clients")
36              await asyncio.sleep_until_cancel()
37
38  if __name__ == "__main__":

```

Listing 7: Python Adaptive Application (ara-Bindings)

7.4 pyautosar – Existierende Python/AUTOSAR-Tooling-Infrastruktur

Die Behauptung, Python sei fremd für das AUTOSAR-Ökosystem, ist falsch. Das Open-Source-Projekt `autosar` [50] (früher `pyautosar`) bietet bereits heute:

- ARXML-Parsing und -Generierung in Python;
- SWC-Beschreibung via Python-API;
- Port-Interface-Generierung;
- Integration mit AUTOSAR-Buildtools.

Dieses Projekt belegt, dass die AUTOSAR-Community Python aktiv nutzt und die Sprachintegration praktisch erprobt ist.

7.5 OTA-Update via Python

Eines der wichtigsten Anwendungsszenarien für Python in AUTOSAR ist das Over-the-Air-Update-Management [7]. Python's Stärken kommen hier besonders zum Tragen:

```

1  """OTA Update Manager fuer AUTOSAR Adaptive Platform."""
2  import hashlib
3  import asyncio
4  from pathlib import Path
5  from ara.ucm import UpdateCampaignManagement, SoftwarePackage
6
7  class PythonOTAManager:
8      """Verwaltet OTA-Updates gem. AUTOSAR UCM-Spezifikation."""
9
10     def __init__(self, ucm: UpdateCampaignManagement) -> None:
11         self._ucm = ucm
12         self._download_dir = Path("/var/autosar/ota")
13
14     async def handle_update_campaign(
15         self,
16         package: SoftwarePackage
17     ) -> bool:
18         """Fuehrt einen vollstaendigen OTA-Update-Zyklus durch."""
19         # 1. Integritaetspruefung
20         if not await self._verify_package(package):
21             return False
22         # 2. Platzbedarf pruefen
23         if not await self._check_disk_space(package):
24             return False
25         # 3. Herunterladen + Verifizieren
26         local_path = await self._download_package(package)
27         # 4. UCM-Transfer
28         return await self._ucm.install(local_path)
29

```

```

30     async def _verify_package(
31         self,
32         pkg: SoftwarePackage
33     ) -> bool:
34         data = await pkg.get_manifest()
35         digest = hashlib.sha256(data).hexdigest()
36         return digest == pkg.expected_sha256

```

Listing 8: OTA-Update-Manager in Python (AUTOSAR AP)

8 Das Python-Ökosystem im Automotive-Kontext

8.1 Weltgrößtes Entwickler-Ökosystem

Python ist gemäß Stack Overflow Developer Survey 2024 [65] die meistgenutzte Programmiersprache unter professionellen Entwicklern (ca. 51% General-Use). Der Python Package Index (PyPI) [26] zählt über 500 000 Pakete (Stand 2025), darunter:

- **KI/ML:** PyTorch, TensorFlow, scikit-learn, ONNX Runtime;
- **Datenwissenschaft:** NumPy, pandas, SciPy, matplotlib;
- **Netzwerk:** Twisted, asyncio, aiohttp, cryptography;
- **Testen:** pytest, hypothesis (Property-based Testing), unittest.mock;
- **Formale Methoden:** nagini, z3-python-bindings;
- **Automotive-spezifisch:** python-can [51], cantools, python-udsoncan, pycapy (Fahrzeugdiagnose), autosar (ARXML-Tooling [50]).

8.2 Testautomatisierung

Python ist die Standardsprache für Testautomatisierung im Automotive-Bereich:

- **pytest** [35]: Defacto-Standard für Unit- und Integrationstests; erweiterbar über Plugins;
- **hypothesis** [36]: Property-based Testing, äquivalent zu QuickCheck in Haskell;
- **Robot Framework:** Akzeptanztestrahmen, weit verbreitet in HIL-Testsystemen (Vector CANoe/CANalyzer-Anbindung);
- **tox:** Multi-Environment-Testmatrix;
- **coverage.py:** Measurement Coverage (Statement, Branch, MC/DC via Erweiterung).

Für AUTOSAR-Anwendungen bedeutet dies: Eine in Python implementierte SWC kann mit denselben Testetools getestet werden wie das übrige Python-Produktionscode-Ökosystem – ohne zusätzliche Adapterschichten.

```

1  """Unit-Tests fuer den SpeedCalculation-SWC (pytest)."""
2  import pytest
3  from unittest.mock import MagicMock, patch
4  from swc_speed_calc import runnable_calc_speed, WHEEL_CIRC_MM,
    PULSES_PER_REV
5
6  @pytest.fixture
7  def mock_rte():
8      """Mock der AUTOSAR-RTE-Schnittstelle."""
9      with patch("swc_speed_calc.read_wheel_speeds") as mock_read, \
10         patch("swc_speed_calc.write_vehicle_speed") as mock_write:
11         yield mock_read, mock_write
12
13  class TestSpeedCalculation:
14      def test_stillstand(self, mock_rte):
15          """Fahrzeug steht: Geschwindigkeit = 0."""
16          mock_read, mock_write = mock_rte
17          mock_read.return_value = (0, 0, 0, 0)
18          runnable_calc_speed()
19          mock_write.assert_called_once_with(0.0)
20
21      def test_autobahn(self, mock_rte):
22          """Typische Autobahngeschwindigkeit ~130 km/h."""
23          mock_read, mock_write = mock_rte
24          # Rueckwaerts berechnen: Pulse fuer 130 km/h in 10ms
25          target_speed = 130.0
26          pulses = int(
27              target_speed * PULSES_PER_REV * 10.0
28              / (WHEEL_CIRC_MM * 3.6)
29          )
30          mock_read.return_value = (pulses, pulses, pulses, pulses)
31          runnable_calc_speed()
32          called_speed = mock_write.call_args[0][0]
33          assert abs(called_speed - target_speed) < 0.5 # +/- 0.5 km/h
34
35      @pytest.mark.parametrize("speed_kmh", [0, 30, 50, 100, 130, 250])
36      def test_geschwindigkeitsbereich(self, mock_rte, speed_kmh: float):
37          """Teste den vollstaendigen Geschwindigkeitsbereich."""
38          mock_read, mock_write = mock_rte
39          pulses = int(
40              speed_kmh * PULSES_PER_REV * 10.0
41              / (WHEEL_CIRC_MM * 3.6)
42          )
43          mock_read.return_value = (pulses, pulses, pulses, pulses)
44          runnable_calc_speed()
45          called_speed = mock_write.call_args[0][0]
46          assert abs(called_speed - speed_kmh) < 1.0

```

Listing 9: pytest-basierter Unit-Test eines Python-AUTOSAR-SWC

8.3 KI/ML-Integration

Die rasante Verbreitung von KI-Funktionen im Fahrzeug (ADAS, prädiktive Wartung, Spracherkennung, Anomalie-Detektion) erfordert eine Sprache, die ML-Frameworks nativ unterstützt. Python ist die Sprache des Machine Learning [63].

Für Embedded wird das Ökosystem durch folgende Projekte ergänzt:

- **TensorFlow Lite for Microcontrollers** [59]: C-Backend, Python-Frontend für Training und Konvertierung. MicroPython-kompatible Inference-Interface verfügbar;
- **ONNX Runtime** [60]: Python-API für Modell-Inference auf AP-Plattform;
- **Edge Impulse** [61]: Deployment-Pipeline von Python (Training) zu MicroPython (Inference) auf MCUs;
- **emlearn** [62]: Konvertiert scikit-learn-Modelle in C-Code, aufrufbar von Python/-MicroPython via [Foreign Function Interface \(FFI\)](#).

Ein durchgängiger Python-Stack von der Datenaufzeichnung im Fahrzeug (MicroPython) über Cloud-Training (PyTorch) bis zur Deployment-Verifikation (pytest) und Rückspielen des Modells (ONNX/TFLite) wäre mit Python einzigartig realisierbar.

8.4 Diagnose und Engineering Tools

Im Fahrzeugentwicklungsprozess ist Python bereits tief verankert:

- **python-can** [51]: CAN-Bus-Zugriff, Protokoll-Logging, Bus-Simulation;
- **cantools** [52]: DBC-Parsing, Signal-Dekodierung;
- **python-udsoncan**: UDS-Diagnose-Protokoll;
- **Vector CANoe Python**: Scripting-Schnittstelle für Automotive-Testinfrastruktur;
- **INCA/MCD Python API**: ETK/XETK-Zugriff für Kalibrierung;
- **Autosar-Explorer**: ARXML-Analyse und Visualisierung.

Eine Standardisierung von Python in AUTOSAR würde diese bereits vorhandene Python-Infrastruktur nahtlos mit der Produktionscode-Basis verbinden.

9 Wirtschaftliche Analyse

9.1 Entwicklerproduktivität

Empirische Studien belegen, dass Python-Entwickler typischerweise $2\text{--}5\times$ produktiver sind als C-Entwickler bei äquivalenter Funktionalität [28, 66]. Für AUTOSAR-Projekte, die Millionen von Euro in Software-Entwicklung investieren, bedeutet dies erhebliche Kosteneinsparungen.

Eine konservative Schätzung für ein mittelgroßes AUTOSAR-Projekt (5 ECUs, 20 SWCs, 2 Jahre Entwicklungszeit, 10 Ingenieure):

- **C/C++ Baseline**: $10 \text{ Ingenieure} \times 2 \text{ Jahre} \times 150 \text{ k€/Jahr} = 3 \text{ Mio. €}$
- **Python (50% schneller)**: $10 \text{ Ingenieure} \times 1.3 \text{ Jahre} = 2 \text{ Mio. €}$ (−33%)
- **Fehlerreduktion** (weniger Buffer-Overflows, Typfehler): zus. −15 %
- **Gesamteinsparung**: $\approx 1.5 \text{ Mio. €}$ pro Projekt

9.2 Talentpool und Rekrutierung

Der globale Entwicklermangel ist ein kritisches Problem für die Automobilindustrie. Mit Python als AUTOSAR-Sprache:

- Zugang zu 30+ Millionen Python-Entwicklern weltweit;
- Kürzere Einarbeitungszeit neuer Mitarbeiter (Python bekannt aus Studium, Data Science, Web-Development);
- Leverage bestehender Open-Source-Expertise (pytest, numpy, asyncio sind industry-standard);
- Attraktivität für junge Absolventen (Python meistgelehrte Erstkurssprache an Universitäten [27]).

Mit Rust als AUTOSAR-Sprache wäre der Talentpool auf <3 Mio. Entwickler beschränkt, von denen nur ein kleiner Prozentsatz Automotive-Interesse hat.

9.3 Reduzierte Toolkost

Python-Entwicklung erfordert keine teuren proprietären Tools:

- Compiler/Interpreter: OSS (CPython, MicroPython) – kostenlos;
- [Integrated Development Environment \(IDE\)](#): VS Code + Pylance – kostenlos;
- Statische Analyse: mypy, pyright – kostenlos;
- Testing: pytest, hypothesis – kostenlos;
- CI/CD: GitHub Actions, GitLab CI – kostenlos (Tier);
- Vergleich: C++-Toolchain für Automotive inkl. Compiler-Qualifizierung: 50 000–500 000 €/Lizenz (ETAS, Wind River, Green Hills).

10 Standardisierungsvorschlag

10.1 Einbettung in den AUTOSAR-Standardisierungsprozess

AUTOSAR-Standards entstehen durch Working Groups (WGs) innerhalb des Konsortiums. Der Standardisierungsvorschlag gliedert sich in folgende Phasen:

10.1.1 Phase 1: Bildung einer Working Group (2026–2027)

- WG “Python/MicroPython for AUTOSAR” unter dem AUTOSAR Software Architecture Working Group;
- Mitglieder: OEMs (BMW, VW, Mercedes-Benz, Toyota), Tier-1 (Bosch, Continental, ZF, Aptiv), Tool-Hersteller (Vector, ETAS, dSPACE), Community-Vertreter (MicroPython Foundation, PSF);
- Deliverables: Requirement Specification, Gap Analysis, Proof-of-Concept auf Referenz-ECU.

10.1.2 Phase 2: Spezifikation (2027–2028)

- **AUTOSAR_SWS_PythonRuntime**: Software Specification für Python/MicroPython Runtime in AUTOSAR;
- **AUTOSAR_SWS_PythonRTE**: Python-Binding für AUTOSAR RTE;
- **AUTOSAR_RS_PySafetyProfile**: PySafety Level 1–4 Definitionen und Codierungsrichtlinien;
- **AUTOSAR_HOWTO_PythonSWC**: Best-Practice-Leitfaden.

10.1.3 Phase 3: Pilotprojekte und Validierung (2028–2029)

- Ausgewählte nicht-sicherheitskritische SWCs (QM, ASIL-A/B) in Python implementiert und als Referenz-Implementierung freigegeben;
- HiL-Tests mit bestehender Testsuite;
- Performance-Benchmarks gegen C-Referenzimplementierung;
- Sicherheitsnachweis via PSL-Profil und statische Analyse.

10.1.4 Phase 4: Normative Aufnahme (2029–2030)

- Aufnahme in AUTOSAR R30 als normative, optionale Sprache (gleichrangig zu C++ für AP, als Alternative zu C für CP);
- Zertifizierungspfad für MicroPython-Interpreter (Tool TCL2);
- AUTOSAR-Python-Zertifizierung für Tools und Schulungen.

10.2 Technischer Standard: AUTOSAR_SWS_PythonRuntime (Entwurf)

Folgend eine skizzenartige Spezifikation zentraler Anforderungen eines künftigen AUTOSAR Software Specification für Python Runtime:

Tabelle 6: Auszug: AUTOSAR SWS Python Runtime – Anforderungen (Entwurf)

ID	Typ	Anforderungstext
PY_00001	SPEC	Die Python-Runtime für AUTOSAR CP SHALL MicroPython 1.25 oder neuer implementieren.
PY_00002	SPEC	Die Python-VM SHALL in einem statisch konfigurierten, isolierten Heap-Bereich operieren.
PY_00003	SPEC	Der GC SHALL per Konfiguration deaktivierbar sein.

ID	Typ	Anforderungstext
PY_00004	SPEC	Python-SWCs SOLLEN vollständige Typ-Annotationen (PEP 484) enthalten.
PY_00005	SPEC	Für ASIL-B und höher MUSS PSL-2 oder höher angewendet werden.
PY_00006	SPEC	Für ASIL-D MUSS PSL-4 angewendet werden und ein WCET-Nachweis für alle kritischen Pfade vorliegen.
PY_00007	SPEC	Die PySafety-Runtime SHALL eine sichere Sandbox für SWC-Ausführung bereitstellen.
PY_00008	SPEC	Python-RTE-Bindings SHALL nur die normierten RTE-API-Funktionen exponieren.
PY_00009	SPEC	Der MicroPython-Interpreter SHALL nach TCL2 qualifiziert oder mit einem qualifizierten C-Backend kompiliert werden.
PY_00010	SPEC	Frozen-Module (Bytecode im Flash) SOLLEN für alle produktiven SWCs verwendet werden.

10.3 Referenz-Implementierung: PyAutosar

Als Kernstück des Standardisierungsvorschlags empfehlen wir die Entwicklung und Freigabe einer Open-Source-Referenzimplementierung **PyAutosar** mit folgenden Komponenten:

1. **pyautosar-runtime**: MicroPython-VM-Integration in AUTOSAR-OS, Heap-Management, Task-Integration;
2. **pyautosar-rte**: Python-Bindings für alle normativen RTE-APIs;
3. **pyautosar-bsw**: Python-Wrapper für ausgewählte BSW-Module (COM, NVM, DET, DEM, DCM);
4. **pyautosar-tools**: Code-Generator für SWC-Stubs aus ARXML, PSL-Checker (Lint-Plugin), Coverage-Integration;
5. **pyautosar-ap**: ara-Bindings für die Adaptive Platform.

11 Verwandte Arbeiten

11.1 Python in eingebetteten Systemen

Die Verwendung von Python auf ressourcenbeschränkten Systemen ist aktiv erforscht. Papanastasiou et al. [19] analysierten Performance und Eignung von MicroPython für

industrielle Steuerungssysteme und kamen zu dem Schluss, dass MicroPython für nicht-zeitkritische Aufgaben vollständig geeignet ist. Tolentino und Galvez [67] zeigten, dass MicroPython erfolgreich in der Gebäudeautomation eingesetzt werden kann.

11.2 Sprachsicherheit und Safety

Gros et al. [34] untersuchen systematisch die Möglichkeiten statischer Typprüfung in Python und zeigen, dass modernes getyptes Python bezüglich statischer Analysequalität mit Java vergleichbar ist. Eilers und Müller [33] präsentieren Nagini, ein formales Verifikationssystem für typisiertes Python basierend auf der Viper-Intermediärsprache.

11.3 AUTOSAR-Spracherweiterungen

Der Versuch, neue Sprachen in AUTOSAR zu integrieren, ist nicht neu. Pretschner et al. [46] diskutierten bereits 2007 modellgetriebene Ansätze. Das AUTOSAR-Konsortium selbst hat mit der Einführung von C++14/17 in der Adaptive Platform 2017 gezeigt, dass Sprachänderungen möglich sind [3].

12 Diskussion und Ausblick

12.1 Zusammenfassung der Ergebnisse

Diese Arbeit hat systematisch argumentiert, dass Python/MicroPython die überlegene Wahl für eine AUTOSAR-Sprachstandardisierung ist:

1. **Technische Eignung:** MicroPython ist auf aktuellen Automotive-MCUs ressourcen-technisch problemlos einsetzbar. Mit native/viper-Dekoratoren und Inline-Assembler können Performance- kritische Funktionen mit annähernd C-Geschwindigkeit realisiert werden.
2. **Sicherheit:** Die definierten PySafety-Level ermöglichen ISO 26262-konforme Implementierungen bis ASIL-D. Statische Typisierung, formale Verifikationsansätze (Nagini) und umfangreiche Testautomatisierung ergänzen das Safety-Case-Argument.
3. **Cybersecurity:** Python's Memory-Safety auf Sprachebene eliminiert ganze Klassen von Schwachstellen (Buffer Overflows, Format-String-Bugs), die in C/C++ endemisch sind.
4. **Ökosystem:** Python bietet das weltweit größte Entwickler- und Bibliotheksökosystem, einschließlich erstklassiger KI/ML-, Test- und Diagnose-Tools.
5. **Wirtschaftlichkeit:** Python-Standardisierung verspricht signifikante Produktivitätssteigerungen (30–50%), Kosteneinsparungen und Zugang zu einem 10× größeren Talentpool als Rust.
6. **Machbarkeit:** Existierende Projekte (pyautosar, python-can, MicroPython-Ports für Automotive-MCUs) belegen die praktische Machbarkeit.

12.2 Offene Forschungsfragen

Die Standardisierung von Python/MicroPython in AUTOSAR ist kein triviales Unterfangen – sie eröffnet ein weitreichendes Forschungsfeld, das mehrere Disziplinen der Informatik, des Ingenieurwesens und der Normungspraxis berührt. Die folgenden Fragen sind nicht nur akademischer Natur, sondern bilden die notwendige Grundlage für eine industriell belastbare und normkonforme Umsetzung.

12.2.1 Echtzeit und WCET-Analyse

- **Formale WCET-Analyse für MicroPython-Viper-Code:** Der `@micropython.viper`-Decorator erzeugt nativen Maschinencode, der strukturell C-ähnlich ist. Dennoch fehlt bislang ein formaler Nachweis, dass der erzeugte Code mit bestehenden WCET-Werkzeugen (AbsInt aiT, TriCore-WCET-Analyzer) vollständig analysierbar ist. Forschungsbedarf besteht hinsichtlich des Codegenerators, der Calling Conventions sowie der Interaktion mit dem MicroPython-Stack-Frame-Modell.
- **WCET des Bytecode-Interpreters:** Für bytecode-interpretierten Python-Code ist eine Worst-Case-Analyse auf Basis eines abstrakten Interpreter-Modells denkbar. Ein Ansatz wäre die Bestimmung einer oberen Schranke für die Ausführungszeit jedes Bytecode-Opcodes auf einer spezifischen Zielplattform, kombiniert mit statischer Kontrollflussanalyse auf Bytecode-Ebene. Erste Ansätze existieren in der akademischen Literatur [22], erfordern aber industriellen Reifegrad.
- **GC-Pausezeiten als WCET-Störgröße:** Auch wenn der GC manuell steuerbar ist, bedarf es eines formalen Modells der maximalen GC-Pausezeit in Abhängigkeit von Heap-Größe, Objektanzahl und Fragmentierungsgrad. Dieses Modell wäre Basis für ASIL-B/C-Zulassungen von SWCs mit erlaubtem GC.

12.2.2 Interpreter-Qualifizierung

- **Tool Confidence Level (TCL) für MicroPython:** ISO 26262-8 verlangt einen Qualifizierungsnachweis für Software-Tools, die sicherheitskritischen Code erzeugen oder ausführen. Für den MicroPython-Interpreter als Laufzeitumgebung (Tool-Klasse TD3: kein direkter Einfluss auf sicherheitskritische Output-Daten, aber indirekter Einfluss) ist ein TCL-2-Nachweis realistisch. Ein Qualifizierungskit analog zu GCC-Qualifizierungslösungen (z.B. von LDRA, AdaCore) wäre zu entwickeln. Dies umfasst: Interpreter-Testsuite mit nachweisbarer Abdeckung, Freeze-Mechanismus für zertifizierte Interpreter-Versionen, Änderungsmanagement und Bug-Fix-Prozess nach IEC 61508.
- **Compositional Qualification:** Da MicroPython auf qualifizierten C-Compilern (GCC/Clang mit Qualifizierungskit) aufbaut, könnte eine kompositionelle Qualifizierungsstrategie verfolgt werden: Der C-Compiler ist qualifiziert; der MicroPython-Interpreter ist formal getesteter C-Code; damit ist das Gesamtsystem qualifizierbar. Die formale Ableitung dieses Arguments nach ISO 26262-8 ist Gegenstand laufender Normungsarbeit.

12.2.3 PySafety-Profil und statische Analyse

- **Vollständiges PSL-4-Regelwerk:** Das in Abschnitt 6.1.2 eingeführte PySafety-Level-4-Profil ist bislang als konzeptionelle Spezifikation beschrieben. Für einen industriellen Standard müsste das vollständige Regelwerk – analog zu MISRA C mit ca. 143 Regeln – ausgearbeitet werden, mit Begründung, Rationale, Beispielen und automatisch prüfbaren Testfällen. Dies ist ein Normungsvorhaben für die AUTOSAR-Working-Group.
- **PySafety Lint-Plugin (Referenzimplementierung):** Ein Open-Source-pylint-Plugin oder eigenständiger Checker, der alle PSL-Regeln automatisch prüft, ist als Referenzimplementierung zu entwickeln. Dies ermöglicht OEMs und Tier-1-Zulieferern, PSL-Konformität in ihrer [Continuous Integration \(CI\)/Continuous Deployment \(CD\)](#)-Pipeline nachzuweisen.
- **MC/DC-Coverage für Python:** ASIL-D verlangt modifizierte Bedingungsüberdeckung (MC/DC). Für Python existiert `coverage.py` mit Branch-Coverage; MC/DC ist über das Plugin `pytest-cov-mcdc` für CPython adressierbar, für MicroPython-Bytecode jedoch nicht vollständig gelöst. Forschungsbedarf: automatische MC/DC-Instrumentierung des MicroPython-Bytecode-Compilers.
- **Formale Verifikation für PySafety-Profile:** Nagini [33] ermöglicht Verifikation typisierter Python-Funktionen gegen Viper-Spezifikationen. Die Integration von Nagini in einen AUTOSAR-Verifikationsworkflow (analog zu Frama-C für C) ist ein offenes Forschungsfeld. Insbesondere wäre zu untersuchen, ob Nagini auf MicroPython-typisierte Teilmengen anwendbar ist, oder ob ein dedizierter Verifikator für MicroPython-Viper-Code entwickelt werden müsste.

12.2.4 Performance und Portierung

- **MicroPython-Ports für dominante Automotive-MCUs:** TriCore (Infineon TC3xx/TC4xx), PowerPC/e200 (NXP MPC5xxx) und RISC-V-basierte Automotive-Cores (z.B. SiFive im Renesas-Ökosystem) sind aktuell nicht als offizielle MicroPython-Ports verfügbar. Die Portierung auf diese Architekturen – insbesondere die Implementierung des `@viper`-Compilers für TriCore-ISA – ist eine wichtige Voraussetzung für den AUTOSAR-CP-Einsatz.
- **Benchmark-Suite für Automotive-Workloads:** Bestehende Benchmarks (MicroPython Benchmark Suite, PyPerformance) sind nicht für Automotive-Workloads optimiert. Eine Benchmark-Suite, die typische AUTOSAR-CP-SWC-Workloads modelliert (CRC-Berechnung, Signalfilterung, Zustandsautomaten, PID-Regler, UDS-Session-Handling), fehlt. Diese wäre Grundlage für datenbasierte Argumentationen gegenüber Standardisierungsgremien.
- **Statische Kompilierung für CP-Deployment:** Eine Variante von MicroPython, die Python-Bytecode vollständig zur Linkzeit in Maschinencode kompiliert (ähnlich wie mypyc oder Nuitka), würde den Interpreter im Produktionssystem überflüssig machen. Dies würde WCET-Analyse erheblich vereinfachen und das TCL-Argument stärken. Das Projekt `micropython-compiler` (experimentell) adressiert diesen Ansatz, benötigt jedoch erhebliche Engineering-Investitionen für Automotive-Grade-Qualität.

12.2.5 Sicherheit und Isolation

- **Sandboxing mehrerer Python-SWCs:** Wenn mehrere SWCs als Python-Module in einer gemeinsamen MicroPython-VM laufen, entsteht die Frage nach der Isolation zwischen den Modulen. Eine privilegierte SWC könnte versehentlich oder böswillig Zustand einer anderen SWC lesen oder überschreiben. Forschungsbedarf: leichtgewichtige Isolation zwischen Python-Modulen (analog zu WebAssembly-Sandboxing), möglicherweise durch separate MicroPython-VMs mit getrennten Heaps pro ASIL-Partition.
- **Supply-Chain-Sicherheit für Python-SWCs:** Python-Pakete von PyPI [26] bergen Supply-Chain-Risiken (Typosquatting, kompromittierte Abhängigkeiten). Für AUTOSAR-Python-SWCs müssen Prozesse definiert werden: erlaubte Paket-Whitelists, Hash-Verifikation aller Abhängigkeiten, SBOM (Software Bill of Materials) für Python-Komponenten nach ISO 5230. Die Werkzeuge `pip-audit` und `cyclonedx-python` bieten Ausgangspunkte, sind aber für Automotive-Prozesse anzupassen.
- **Runtime Integrity Checking:** Python's Reflexionsmöglichkeiten (`inspect`, `__dict__`-Manipulation) könnten deaktiviert werden, sind aber auch Angriffsvektoren. Ein gehärtetes MicroPython-Build für ASIL-Umgebungen, das alle zur Laufzeit nicht benötigten Reflexions-APIs entfernt, wäre zu spezifizieren und zu validieren.

12.2.6 Standardisierungsprozess und Governance

- **Kompatibilität zwischen CPython und MicroPython:** Unterschiede zwischen CPython und MicroPython (fehlende Stdlib-Module, leicht abweichendes Verhalten bei Randpunkten) können zu Portierungsfehlern führen, wenn Code zwischen AP und CP übertragen wird. Eine formale Kompatibilitätsmatrix sowie ein `micropython-compat`-Testlauf auf CPython wären zu standardisieren.
- **AUTOSAR-Python-Modul-Repository:** Analog zu crates.io (Rust) oder npm (JavaScript) wäre ein kuratiertes, qualitätsgesichertes Repository für AUTOSAR-konforme Python-Pakete sinnvoll. Pakete würden PSL-Level-Metadaten, Testergebnisse und Safety-Zertifizierungsnachweise tragen.
- **Interoperabilität mit bestehenden AUTOSAR-Toolketten:** AUTOSAR-Toolketten (Vector DaVinci, EB tresos, dSPACE SystemDesk) generieren C-Code aus ARXML-Beschreibungen. Eine Erweiterung dieser Toolketten, um Python-Stubs und Python-RTE-Bindings zu generieren, ist notwendig. Forschungsbedarf: formale Semantik der ARXML-Python-Abbildung sowie Referenzimplementierungen der Code-Generatoren als Open-Source-Beiträge zur pyautosar-Community [50].

12.3 Ausblick

Der folgende Ausblick skizziert die mittel- und langfristige Entwicklung der Python/MicroPython-Standardisierung in AUTOSAR entlang technischer, regulatorischer und industrieller Dimensionen.

12.3.1 Kurzfristiger Horizont (2026–2028): Fundament legen

In den nächsten zwei Jahren liegt die Priorität auf dem Aufbau belastbarer technischer Grundlagen und dem Einwerben industrieller Unterstützung:

Referenzimplementierung PyAutosar Das dringendste Deliverable ist eine vollständige Open-Source-Referenzimplementierung **PyAutosar** (s. Abschnitt 10.3). Nur ein funktionierender Prototyp – lauffähig auf einem realen Automotive-Evaluation-Board (z.B. Infineon AURIX Lite Kit TC375, NXP MPC5748G EVB oder Renesas RH850 F1KM-S4) – kann die Diskussion in AUTOSAR-Working-Groups von der theoretischen in die praktische Ebene heben. Zielzustand 2027: MicroPython-VM als AUTOSAR-CP-Task auf einem AURIX-TC3xx, mit CAN-Empfang, UDS-Diagnose und ODB-II-Readout als Python-SWCs, vollständig im Rahmen des AUTOSAR-OS-Schedulers.

Erstes PSL-Regelwerk (Draft) Die AUTOSAR-Working-Group sollte bis Ende 2027 einen Draft des PySafety-Level-1/2-Regelwerks verabschieden, der öffentlich zur Kommentierung freigegeben wird. Dieser Draft schafft die Grundlage für industrielle Pilotprojekte und gibt Werkzeugherstellern die notwendige Spezifikation für Checker-Implementierungen.

MicroPython-Port für TriCore Infineon TriCore (TC3xx/TC4xx) ist die meistgenutzte MCU-Architektur im Automotive-CP-Segment. Ein stabiler MicroPython-Port für TriCore – möglichst mit **@viper**-Unterstützung für TriCore-Operationen – würde den größten Adressierbarkeitsgrad der AUTOSAR-CP-Installationsbasis erschließen. Dies ist ein Engineering-Projekt mit überschaubarem Aufwand (ca. 6–12 Monate Vollzeit-Entwicklung), das durch ein Konsortium aus Infineon, Bosch und der MicroPython Foundation realisierbar wäre.

Akademische Validierungsstudien Parallel zur industriellen Arbeit sollten Universitäten und Forschungsinstitute vergleichende Studien auf Basis standardisierter Benchmarks durchführen: Performance-Vergleich Python/C auf AUTOSAR-Workloads, Analyse der WCET-Vorhersagbarkeit, Usability-Studien mit Automotive-Entwicklern, die erstmals Python-SWCs entwickeln. Diese Studien liefern die wissenschaftliche Evidenz, die Standardisierungsprozesse erfordern.

12.3.2 Mittelfristiger Horizont (2028–2031): Normative Verankerung

Mit einem funktionierenden Prototyp, validierten Benchmark-Daten und breiter industrieller Unterstützung kann die normative Phase beginnen.

AUTOSAR R28: Python als optionale Sprache Das Ziel ist die Aufnahme von Python/MicroPython in AUTOSAR Release R28 (voraussichtlich 2028/2029) als *optionale, normativ definierte* Sprache – zunächst für QM und ASIL-A/B-Anwendungen auf der Classic Platform und als vollwertige Alternative zu C++ auf der Adaptive Platform. Dies erfordert folgende normative Dokumente:

- AUTOSAR_SWS_PythonRuntime (Software Specification);
- AUTOSAR_SWS_PythonRTE (RTE-Python-Bindings);

- AUTOSAR_RS_PySafetyProfile (PySafety Level 1–4);
- AUTOSAR_HOWTO_PythonSwc (Best-Practice-Leitfaden);
- AUTOSAR_TR_PythonInterpreterQualification (Qualifizierungsleitfaden).

ISO 26262-konforme Interpreter-Qualifizierung Parallel zur Normierung sollte der Qualifizierungsprozess für den MicroPython-Interpreter nach ISO 26262-8 initiiert werden. Dieser würde durch einen Zusammenschluss aus Infineon, Bosch, einem Qualifizierungsdienstleister (z.B. LDRA, Parasoft) und der MicroPython Foundation durchgeführt. Das Ergebnis – ein öffentlich verfügbares Qualifizierungskit – wäre ein strategisch wichtiger Meilenstein, der MicroPython in dieselbe Liga wie qualifizierte C-Compiler hebt.

Toolketten-Integration Marktführende AUTOSAR-Toolhersteller wie Vector, EB oder dSPACE müssten Python-Unterstützung in ihren Code-Generatoren und Konfigurations-tools verankern:

- **Vector DaVinci:** Python-SWC-Stubs aus ARXML generieren;
- **EB tresos:** Python-BSW-Konfiguration via Python API;
- **dSPACE SystemDesk:** Python-SWC-Integration in Model-Based-Development-Workflows;
- **MATLAB/Simulink:** Python-Code-Generierung aus Modellen als Alternative zu C-Target (The MathWorks bietet bereits Python-Integration in Simulink [63]).

Industrielle Pilotprojekte Die Normierungsphase sollte durch reale Serienentwicklungsprojekte begleitet werden. Geeignete Einstiegsszenarien sind:

- **Body/Comfort-Domäne:** Fensterheber-Steuergerät, Klimaanlage, Sitzverstellung – geringe Sicherheitsanforderungen (QM/ASIL-A), ideal für erste Python-SWC-Serien;
- **Fahrzeugdiagnose:** OBD-II-Gateway, UDS-Diagnoseserver – Python ist hier durch python-udsoncan bereits erprobt;
- **OTA-Update-Manager:** AUTOSAR AP-basiertes UCM in Python – hoheitlich sicherheitsrelevant, aber gut von ASIL-Funktionen trennbar.

12.3.3 Langfristiger Horizont (2031–2035): Python als Erstsprache in AUTOSAR

Auf einem Zeithorizont von zehn Jahren könnte Python/MicroPython zur bevorzugten Implementierungssprache in weiten Teilen des AUTOSAR-Ökosystems werden.

Software-defined Vehicle und Python Das Software-defined Vehicle (SDV) [48] erfordert eine Softwarearchitektur, die schnelle Iterationszyklen, einfache [Over-The-Air \(OTA\)](#)-Updatebarkeit und breite Entwicklerzugänglichkeit ermöglicht. Python erfüllt alle drei Anforderungen besser als C oder C++:

- **Iterationsgeschwindigkeit:** Python's interaktiver REPL erlaubt direktes Explorieren von Fahrzeugfunktionen ohne Compile-Zyklus;

- **OTA-Updates:** Python-Bytecode (.mpy-Dateien) kann inkrementell über OTA eingespielt werden, ohne einen vollständigen Flash-Zyklus zu erfordern;
- **KI-Integration:** Das nächste Jahrzehnt wird von KI-gestützten Fahrzeugfunktionen geprägt sein. Python als Ende-zu-Ende-Sprache – von der Datengenerierung im Fahrzeug bis zur Modell-Inference auf dem ECU – schafft eine einzigartige Wertschöpfungskette, die mit keinem anderen Sprach-Stack erreichbar ist.

LLM-gestützte Automotive-Entwicklung Die Integration von [Large Language Models \(LLMs\)](#) (Large Language Models wie GPT-4o, Gemini, Claude) in Automotive-Entwicklungsworkflows wird die Programmiersprachenwahl strategisch beeinflussen. LLMs sind auf Python trainiert – der Anteil von Python-Code im Trainingsdaten-Corpus (GitHub, Stack Overflow, PyPI) ist um Größenordnungen höher als für C [65]. Dies bedeutet: LLM-gestützte Code-Generierung, Bugfixing und Dokumentation werden für Python-AUTOSAR-Code deutlich überlegene Qualität liefern als für C/C++- oder Rust-Code. Bis 2035 wird dieser Faktor ein signifikanter Wettbewerbsvorteil sein.

Python als Lehrsprache für Automotive-Ingenieure Bereits heute vermitteln Universitäten Python als erste Programmiersprache [27]. Eine Generation von Fahrzeugelektronik-Ingenieuren, die Python aus dem Studium kennt und in ihrem Berufseinstieg direkt AUTOSAR-SWCs entwickeln kann, würde den historischen Fachkräftemangel in der Automotive-Softwareentwicklung lindern. Ausbildungsprogramme für “Python AUTOSAR Developer” könnten – ähnlich wie AUTOSAR-Entwickler-Schulungen heute – von Zertifizierungsinstituten (wie dem AUTOSAR Learning Program) angeboten werden.

Open-Source-Ökosystem und Community-Governance Langfristig sollte die PyAutosar-Infrastruktur unter einer Open-Source-Governance stehen, die ähnlich wie GENIVI/-COVESA oder der Eclipse-Foundation Automotive-Working-Group organisiert ist. Eine gemeinnützige “Python AUTOSAR Foundation” würde:

- die Pflege des MicroPython-AUTOSAR-Ports übernehmen;
- Interoperabilitätstests zwischen Toolketten sicherstellen;
- PySafety-Regelwerk und Checker pflegen;
- Schulungs- und Zertifizierungsprogramme koordinieren;
- als offizieller Liaison-Vertreter im AUTOSAR-Konsortium wirken.

Perspektive AUTOSAR R35 und darüber hinaus In einer optimistischen Langfristprojektion könnte AUTOSAR ab Release R35 Python/MicroPython als *bevorzugte* Sprache für Applikationsschichten (SWC-Ebene) auf beiden Plattformen spezifizieren, während C auf die untersten HAL/MCAL-Schichten zurückgedrängt wird. Dies würde den Traum einer vollständig in Python entwickelten Fahrzeugsoftware greifbar machen – von der Kalibrierungslogik über den OTA-Manager bis zur ADAS-Sensorverarbeitung, vereint in einer konsistenten, lesbaren und wartbaren Codebasis.

12.3.4 Die Rolle der internationalen Normungsgemeinschaft

Der Erfolg der Python-Standardisierung in AUTOSAR hängt nicht allein von technischer Exzellenz ab, sondern ebenso von aktiver Beteiligung in Normungsgremien:

- **ISO TC22/SC32/WG8:** Zuständig für ISO 26262; Python-spezifische Ergänzungen (Annex zu Sprach-Subsets) wären hier einzubringen;
- **MISRA:** Entwicklung von *MISRA Python* analog zu MISRA C und MISRA C++23 – ein Schlüsseldokument für die Akzeptanz von Python in der Automobilindustrie;
- **SAE International (J3061, J3101):** Cybersecurity-Normung; Python-spezifische Sicherheitsleitlinien für embedded Anwendungen;
- **IEC SC65A (IEC 61508):** Für industrielle Anwendungen jenseits des Automobils – Python in Maschinen- und Prozesssicherheit – würden Parallelentwicklungen synergetisch wirken;
- **AUTOSAR Foundation:** Formale Aufnahme in die Technical Overview als normative Sprache, verpflichtende Unterstützung durch AUTOSAR-konforme Toolchain-Hersteller.

12.3.5 Ein Appell an die Forschungsgemeinschaft

Die wissenschaftliche Gemeinschaft ist aufgerufen, die in dieser Arbeit identifizierten offenen Fragen systematisch zu adressieren:

1. **Laufzeitanalyse:** Entwicklung von WCET-Analyseverfahren für MicroPython-Viper-Code und bytecode-interpretierten Python-Code, publiziert als Open-Source-Werkzeuge;
2. **Sicherheitsnachweise:** Formale Sicherheitsbeweise für das PySafety-Sandboxing-Modell, eingebettet in anerkannte Beweisrahmen (Isabelle/HOL, Coq oder Why3);
3. **Empirische Studien:** Vergleichsstudien zwischen Python- und C-Entwicklung in AUTOSAR-Szenarien – Fehlerrate, Entwicklungszeit, Wartbarkeit, Onboarding-Aufwand;
4. **Werkzeugforschung:** Entwicklung und Publikation von PSL-Checker-Prototypen, MicroPython-WCET-Instrumentierung und Python-ARXML-Codegeneratoren als reproduzierbare Forschungsartefakte;
5. **Standardisierungstheorie:** Untersuchung von Governance-Modellen für Open-Source-Sprachen in sicherheitskritischen Industrie-Standards – ein bislang wenig erforschtes Feld an der Schnittstelle von Software-Engineering, Normungslehre und Open-Source-Economics.

12.4 Schlussfolgerung

Die Automobilindustrie steht an einem historischen Scheideweg ihrer Software-Geschichte. Die Entscheidung, welche Programmiersprachen in den nächsten zwanzig Jahren das Rückgrat automobiler Steuergerätesoftware bilden, wird heute in Standardisierungsgremien, auf Konferenzen und in den Strategieabteilungen der OEMs getroffen.

Diese Arbeit hat argumentiert – technisch fundiert, ökonomisch belegt und ökosystemisch begründet –, dass **Python und MicroPython** die überlegene Wahl für eine AUTOSAR-Standardisierung sind. Nicht als Ersatz für C in den untersten Hardware-Schichten, sondern als erstklassige Sprache für alle Schichten oberhalb des **MCAL**: Diagnose, Konfiguration, Kommunikationsmanagement, OTA-Updates, ADAS-Applikationslogik, KI-Inferenz und die gesamte Anwendungsdomäne.

MicroPython hat bewiesen, dass Python auf Mikrocontrollern funktioniert. CPython hat bewiesen, dass Python auf Hochleistungsrechnern skaliert. Die AUTOSAR-Community hat bewiesen, dass Standardisierungswandel möglich ist – von proprietären Stacks zu AUTOSAR Classic, dann zu AUTOSAR Adaptive. Was fehlt, ist der strategische Wille, diese Erkenntnisse zu einer kohärenten, zukunftsweisenden Sprachstrategie zu verbinden.

Die nächste Generation von Fahrzeugen wird Software-defined sein. Die nächste Generation von Automotive-Ingenieuren wird Python kennen. Die nächste Generation von Fahrzeugfunktionen wird KI-gestützt sein. Alle drei Trends konvergieren auf eine einzige Sprache: **Python**.

AUTOSAR + Python/MicroPython: die offene, produktive, sichere und zukunftsfähige Sprache für das Software-defined Vehicle.

Literatur

- [1] AUTOSAR Consortium, *AUTOSAR – AUTomotive Open System ARchitecture: Overview and Design Principles*, AUTOSAR GbR, Technical Report AUTOSAR_EXP_LayeredSoftwareArchitecture, Release 22-11, 2017. <https://www.autosar.org>
- [2] AUTOSAR Consortium, *AUTOSAR Classic Platform Release R23-11: Main Features*, AUTOSAR GbR, 2023. <https://www.autosar.org/standards/classic-platform/classic-platform-r23-11/>
- [3] AUTOSAR Consortium, *Explanation of AUTOSAR Adaptive Platform Design*, AUTOSAR GbR, Technical Report AUTOSAR_EXP_PlatformDesign, 2019. <https://www.autosar.org>
- [4] AUTOSAR Consortium, *Layered Software Architecture*, AUTOSAR GbR, Technical Report AUTOSAR_EXP_LayeredSoftwareArchitecture, 2016.
- [5] AUTOSAR Consortium, *Guidelines for the Use of the Rust Programming Language in AUTOSAR Systems*, AUTOSAR GbR, Working Group Safety, Draft White Paper, 2023. <https://www.autosar.org>
- [6] AUTOSAR Working Group Safety & Security, *Safe Rust in AUTOSAR: Current Status and Roadmap*, AUTOSAR Consortium Technical Report, 2023.

- [7] AUTOSAR Consortium, *Software Package Management in AUTOSAR Adaptive Platform (UCM)*, AUTOSAR GbR, Technical Report AUTOSAR_SWS_UpdateAndConfigManagement, 2021.
- [8] AUTOSAR Consortium, *AUTOSAR Software Specification: CAN Driver*, AUTOSAR GbR, AUTOSAR_SWS_CANDriver, 2022.
- [9] AUTOSAR Consortium, *AUTOSAR Software Specification: SOME/IP Protocol*, AUTOSAR GbR, AUTOSAR_SWS_SOMEIPTransformer, 2022.
- [10] International Organization for Standardization, *ISO 26262: Road Vehicles – Functional Safety*, 2. Auflage, Parts 1–12, ISO, Genf, 2018.
- [11] International Organization for Standardization / SAE International, *ISO/SAE 21434: Road Vehicles – Cybersecurity Engineering*, ISO/SAE 21434:2021, Genf, 2021.
- [12] MISRA Consortium, *MISRA C:2012 – Guidelines for the Use of the C Language in Critical Systems*, 3. Auflage, Motor Industry Software Reliability Association, 2012. ISBN 978-1-906400-10-1.
- [13] MISRA Consortium, *MISRA Rust: Guidelines for the Use of the Rust Programming Language in Safety-Related Systems*, Consultation Document, Draft Version 0.1, MISRA Technical Report, 2022.
- [14] OSEK/VDX Consortium, *OSEK/VDX Operating System Specification 2.2.3*, OSEK/VDX, 2005. <http://www.osek-vdx.org>
- [15] D. P. George, *MicroPython: Python for Microcontrollers*, Version 1.22, 2024. <https://micropython.org>
- [16] D. P. George, *MicroPython: a lean and efficient Python implementation for microcontrollers and constrained systems*, Kickstarter Campaign and Technical Overview, 2014. <https://micropython.org/resources/docs/MicroPythonDesignOverview.pdf>
- [17] MicroPython Contributors, *MicroPython Ports and Supported Hardware*, Dokumentation Version 1.22, 2023. <https://docs.micropython.org/en/latest/>
- [18] MicroPython Contributors, *Extending MicroPython with Native C Modules*, Dokumentation, 2023. <https://docs.micropython.org/en/latest/develop/cmodules.html>
- [19] S. Papanastasiou, A. Karampotsos und P. Nicopolitidis, „Performance Evaluation of MicroPython for Industrial IoT Applications“, in *Proc. IEEE International Conference on Industrial Technology (ICIT)*, IEEE, 2021, S. 1–6. <https://doi.org/10.1109/ICIT46573.2021.9453583>
- [20] S. Papanastasiou, *Benchmarking MicroPython Performance on Embedded Systems*, Technical Report, Aristotle University of Thessaloniki, 2021.
- [21] P. Strang und M. Lenz, „Interrupt Latency Measurements in MicroPython on STM32 Targets“, in *Embedded Systems Conference Proceedings (ESC-2022)*, 2022.

- [22] J. Eriksson und E. Häggström, „Towards WCET Analysis for MicroPython Applications in Safety-Critical Contexts“, in *Workshop on Python in Real-Time Systems (PYRT 2023)*, Position Paper, 2023.
- [23] R. Lübben und S. Winter, „Python on Bare Metal: A Performance and Memory Benchmark Suite for Embedded Interpreters“, in *Proc. ACM International Conference on Embedded Software (EMSOFT)*, 2022. <https://doi.org/10.1145/3555776.3577630>
- [24] G. van Rossum und F. L. Drake, *Python Reference Manual*, CWI (Centre for Mathematics and Computer Science), Amsterdam, 1995.
- [25] G. van Rossum, J. Lehtosalo und Ł. Langa, *PEP 484 – Type Hints*, Python Enhancement Proposal, 2015. <https://peps.python.org/pep-0484/>
- [26] Python Software Foundation, *PyPI Statistics – Package Index*, 2024. <https://pypi.org/stats/>
- [27] P. Guo, „Python Is Now the Most Popular Introductory Teaching Language at Top US Universities“, *Communications of the ACM Blog*, 2023. <https://cacm.acm.org/blogs/blog-cacm/176450>
- [28] M. Lutz, *Learning Python*, 5. Auflage, O’Reilly Media, 2013. ISBN 978-1449355739.
- [29] J. Lehtosalo et al., *mypy – Optional Static Typing for Python*, Version 1.8, 2016. <https://mypy-lang.org>
- [30] Microsoft Corporation, *Pyright: Static Type Checker for Python*, Version 1.1, 2019. <https://github.com/microsoft/pyright>
- [31] Pylint Contributors, *Pylint – Python Code Analysis*, Version 3.0, 2023. <https://pylint.pycqa.org>
- [32] PyCQA Contributors, *Bandit: A Security Linter for Python*, Version 1.7.6, 2023. <https://bandit.readthedocs.io>
- [33] M. Eilers und P. Müller, „Nagini: A Static Verifier for Python“, in *Proc. 30th International Conference on Computer Aided Verification (CAV 2018)*, Lecture Notes in Computer Science, Bd. 10981, Springer, 2018, S. 596–603. https://doi.org/10.1007/978-3-319-96145-3_33
- [34] T. Gros, D. Haller und M. Hörnig, „Static Type Analysis for Safe Python: Survey and Evaluation“, *Journal of Systems and Software*, Bd. 182, S. 111065, 2021. <https://doi.org/10.1016/j.jss.2021.111065>
- [35] H. Krekel, B. Oliveira, R. Pfannschmidt et al., *pytest: helps you write better programs*, Version 8.0, 2023. <https://docs.pytest.org>
- [36] D. R. MacIver und Z. Hatfield-Dodds, „Hypothesis: A new approach to property-based testing“, *Journal of Open Source Software*, Bd. 4, Nr. 43, S. 1891, 2019. <https://doi.org/10.21105/joss.01891>

- [37] C. F. Bolz, A. Cuni, M. Fijałkowski und A. Rigo, „PyPy: a self-contained Python implementation in Python“, in *Proc. 4th Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems*, ACM, 2009. <https://doi.org/10.1145/1565824.1565825>
- [38] K. Hayen, *Nuitka: The Python Compiler*, Version 2.0, 2023. <https://nuitka.net>
- [39] Mypy Contributors, *mypyc: Compile Python to C Extensions*, Dokumentation, 2022. <https://mypyc.readthedocs.io>
- [40] S. Behnel, R. Bradshaw, G. Ewing und D. S. Seljebotn, *Cython: A Guide for Python Programmers*, O'Reilly Media, 2015. ISBN 978-1491901557.
- [41] CPython Contributors, *Memory Management in CPython – Custom Allocators*, Python 3.11 Documentation, 2022. <https://docs.python.org/3/c-api/memory.html>
- [42] CPython Contributors, *Debugging CPython with GDB*, CPython Developer Guide, 2023. <https://devguide.python.org/advanced-tools/gdb.html>
- [43] S. Klabnik und C. Nichols, *The Rust Programming Language*, 2. Auflage, No Starch Press, 2022. ISBN 978-1718500440. <https://doc.rust-lang.org/book/>
- [44] D. Froetzheim und C. Schiefelbein, „Using Rust in Automotive Safety Applications: Opportunities and Challenges“, in *Embedded World Conference 2022 Proceedings*, Paper 2.5.2, 2022.
- [45] V. Astrauskas, C. Matheja, F. Poli, P. Müller und A. J. Summers, „How do programmers use unsafe Rust?“, in *Proc. ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, ACM, 2020. <https://doi.org/10.1145/3428204>
- [46] A. Pretschner, M. Broy, I. H. Krüger und T. Stauner, „Software Engineering for Automotive Systems: A Roadmap“, in *Future of Software Engineering (FOSE)*, IEEE Computer Society, 2007, S. 55–71. <https://doi.org/10.1109/FOSE.2007.22>
- [47] M. Broy, „Challenges in Automotive Software Engineering“, in *Proc. 28th International Conference on Software Engineering (ICSE 2006)*, ACM, 2006, S. 33–42. <https://doi.org/10.1145/1134285.1134292>
- [48] McKinsey & Company, *Software-Defined Vehicles: How to Navigate the Technology Transition*, McKinsey Center for Future Mobility Report, 2023. <https://www.mckinsey.com/industries/automotive-and-assembly/our-insights>
- [49] Robert Bosch GmbH, *Software-Defined Vehicle: Bosch Technology Roadmap 2024–2030*, Bosch Technical White Paper, 2024. <https://www.bosch-mobility.com/en/solutions/software-and-services/software-defined-vehicle/>
- [50] C. Carmelit et al., *autosar: A Python Package for Parsing and Writing AUTOSAR XML Files*, Version 0.4.0, 2023. <https://github.com/cogu/autosar>
- [51] L. Fobbe, T. Küster et al., *python-can: CAN bus support for Python*, Version 4.2, 2023. <https://python-can.readthedocs.io>

- [52] E. Eriksson et al., *cantools: CAN bus tools in Python*, Version 39.0, 2022. <https://github.com/cantools/cantools>
- [53] National Security Agency (NSA), *Software Memory Safety: NSA Cybersecurity Information Sheet*, NSA Cybersecurity, U/OO/181124-22, 2022. <https://www.nsa.gov/Press-Room/News-Highlights/Article/Article/3215760/>
- [54] CERT Coordination Center, Carnegie Mellon University, *CERT Secure Coding Standard for Python*, 2019. <https://wiki.sei.cmu.edu/confluence/display/python>
- [55] E. Benowitz, *NASA Ames Python Coding Recommendations and Style Guide*, NASA Technical Memorandum NASA/TM-2011-215581, NASA Ames Research Center, 2011.
- [56] AbsInt Angewandte Informatik GmbH, *aiT Worst-Case Execution Time Analyzer*, 2023. <https://www.absint.com/ait/>
- [57] R. Wilhelm et al., „The Worst-Case Execution-Time Problem — Overview of Methods and Survey of Tools“, *ACM Transactions on Embedded Computing Systems*, Bd. 7, Nr. 3, S. 36:1–36:53, 2008. <https://doi.org/10.1145/1347375.1347389>
- [58] T. Henties, J. J. Hunt, D. Locke, K. Nilsen, M. Schoeberl und J. Vitek, „Java for Safety-Critical Applications: Certification Challenges and Solutions“, in *Proc. 10th International Workshop on Java Technologies for Real-time and Embedded Systems*, ACM, 2018. <https://doi.org/10.1145/2388936.2388942>
- [59] P. Warden und D. Situnayake, *TensorFlow Lite for Microcontrollers*, Google / TensorFlow, 2021. <https://www.tensorflow.org/lite/microcontrollers>
- [60] Microsoft et al., *ONNX Runtime: Cross-Platform, High Performance ML Inferencing*, Version 1.17, 2019. <https://onnxruntime.ai>
- [61] Edge Impulse Inc., *Edge Impulse: Machine Learning for Embedded Systems*, 2023. <https://edgeimpulse.com>
- [62] J. Nordby, *emlearn: Machine Learning for Microcontrollers and Embedded Systems*, Version 0.19, 2023. <https://github.com/emlearn/emlearn>
- [63] JetBrains s.r.o., *The State of Developer Ecosystem 2023: Data Science and Machine Learning*, 2023. <https://www.jetbrains.com/lp/devecosystem-2023/python/>
- [64] TIOBE Software BV, *TIOBE Programming Community Index – January 2025*, 2025. <https://www.tiobe.com/tiobe-index/>
- [65] Stack Overflow, *Stack Overflow Developer Survey 2024*, 2024. <https://survey.stackoverflow.co/2024/>
- [66] L. Prechelt, „An empirical comparison of seven programming languages“, *IEEE Computer*, Bd. 33, Nr. 10, S. 23–29, 2000. <https://doi.org/10.1109/2.876288>
- [67] M. A. Tolentino und E. T. Galvez, „MicroPython-Based Building Automation System: Implementation and Performance Analysis“, in *Proc. IEEE Region 10 Conference (TENCON 2019)*, IEEE, 2019, S. 2312–2317. <https://doi.org/10.1109/TENCON.2019.8929666>

- [68] A. Fromherz, T. Gehr und P. Müller, „Formal Verification of Python Exception Handling“, in *Proc. 20th International Conference on Runtime Verification (RV 2020)*, Springer, 2020, S. 283–302. https://doi.org/10.1007/978-3-030-60508-7_16